

1 Appendix U: Agent Semantics

U.1. Scope and platform status

This appendix defines the semantics of agents used in the search, platform, proof-management, and audit layers of the AK framework. It belongs to the Search / Platform Appendices. It does not enlarge the Core mathematical package of Chapters 1–8.

The central rule of this appendix is:

Agent output is not proof unless verified by the declared verifier-side protocol.

In particular, no human proposal, AI-generated answer, search score, confidence value, ranking, heuristic, proof sketch, platform artifact, or stored object may replace:

$$B\text{-Gate}^+, \quad \text{Eval}_{1,W,\tau}(F) = 0, \quad C_{\tau,W}F, \quad \phi_{i,W,\tau}, \quad (\mu_{\text{Collapse}}, u_{\text{Collapse}}) = (0, 0), \quad \Sigma\delta < \text{gap}_{\tau}.$$

Remark 1.1 (Search / Platform status). This appendix describes who or what may generate, transform, inspect, verify, store, or audit objects in the AK workflow. It does not create new mathematical theorems.

Remark 1.2 (Relation to Core sovereignty). Chapter 6 established Core sovereignty. The same rule governs this appendix: agents may propose, navigate, annotate, package, or criticize data, but only verifier-side Core conditions determine Core pass/reject.

Remark 1.3 (Why agent semantics are needed). The AK framework is search-heavy and platform-heavy. It may involve human judgment, AI generation, automated computation, proof assistants, artifact stores, and certificate graphs. Without explicit agent semantics, candidate outputs can be accidentally inflated into claims. This appendix prevents that inflation.

U.2. Agent types

Definition 1.4 (Agent). An *agent* is any human, AI system, automated tool, proof assistant component, search module, platform service, or hybrid workflow component that performs an action in the AK search, platform, proof-management, or audit pipeline.

Definition 1.5 (Human agent). A *human agent* is a person who proposes, reviews, edits, verifies, audits, approves, rejects, or interprets AK-related objects or claims.

Definition 1.6 (AI agent). An *AI agent* is a machine-learning, language-model, symbolic, neuro-symbolic, or automated reasoning component that proposes, transforms, summarizes, searches, ranks, critiques, or assists with AK-related objects or claims.

Definition 1.7 (Tool agent). A *tool agent* is a deterministic or semi-deterministic software component, such as a persistence-computation engine, proof assistant, type checker, parser, build system, replay engine, artifact validator, dependency resolver, or certificate checker.

Definition 1.8 (Platform agent). A *platform agent* is a service that stores, versions, indexes, retrieves, displays, routes, or synchronizes AK artifacts, proof objects, logs, manifests, or certificate records.

Definition 1.9 (Hybrid agent). A *hybrid agent* is a workflow in which human, AI, tool, and platform agents jointly produce, transform, inspect, verify, or audit an object.

Remark 1.10 (Agent type does not determine validity). The type of agent that produced an object does not determine the object’s proof status. A human-written claim may be false. An AI-generated lemma may be correct. A tool-generated artifact may be misconfigured. A stored artifact may be obsolete. Validity depends on verifier-side checks, not on authorship.

U.3. Agent roles

Definition 1.11 (Proposer). A *proposer* is an agent that suggests a candidate object, lemma, bridge, window, threshold, realization, proof route, diagnostic interpretation, certificate record, or proof object field.

Definition 1.12 (Searcher). A *searcher* is an agent that explores a search space and returns candidate regions, structures, windows, bridges, proof fragments, counterexamples, or artifact references.

Definition 1.13 (Mapper). A *mapper* is an agent that organizes candidate objects into maps, diagrams, dependency graphs, validity maps, bridge maps, or certificate DAGs.

Definition 1.14 (Lifter). A *lifter* is an agent that attempts to turn a candidate, advisory object, search artifact, or platform artifact into a Core-readable object, interface object, formal fragment, or certificate candidate.

Definition 1.15 (Verifier). A *verifier* is an agent or procedure that checks whether a declared object, formal fragment, bridge, certificate record, gate, diagnostic, ledger, or proof object satisfies the relevant verifier-side criteria.

Definition 1.16 (Auditor). An *auditor* is an agent that checks whether the verification process, artifact references, manifest entries, proof object schema, provenance records, and replay data are sufficient for independent reconstruction.

Definition 1.17 (Curator). A *curator* is an agent that stores, labels, versions, indexes, maintains, or retires proof objects, artifacts, status labels, and platform records.

Definition 1.18 (Adjudicator). An *adjudicator* is an agent that resolves status conflicts among outputs, for example when one agent labels an object as a candidate while another reports a failure. Adjudication does not establish mathematical validity unless backed by verifier-side data.

Remark 1.19 (Roles may overlap). A single agent may act in multiple roles. For example, an AI system may propose a bridge, a human may edit it, and a proof assistant may verify a formal fragment. However, the semantic role of each action must remain distinguishable.

U.4. Proposer–verifier separation

Definition 1.20 (Proposer-side data). *Proposer-side data* are outputs generated before verifier-side validation. They include:

- (i) candidate lemmas;
- (ii) candidate proof routes;
- (iii) advisory indicators;
- (iv) search rankings;
- (v) AI confidence values;
- (vi) validity maps;
- (vii) bridge program sketches;
- (viii) informal explanations;
- (ix) draft certificate records;

- (x) candidate normalization or realization routes;
- (xi) candidate [Spec] bridges;
- (xii) proposed proof object fields.

Definition 1.21 (Verifier-side data). *Verifier-side data* are the data used to determine verified status, Core pass/reject, or audit-readable acceptance. They include:

- (i) declared objects;
- (ii) declared windows W ;
- (iii) threshold and collapse policies;
- (iv) repaired evaluation records

$$\text{Eval}_{i,W,\tau}(F) = T_{\tau}^{\text{bar}}(W_W \mathbf{P}_i(F));$$

- (v) admissible representative records

$$C_{\tau,W}F$$

when invoked;

- (vi) eligibility records when the Ext^1 -clause is invoked;
- (vii) declared tower comparison artifacts

$$\phi_{i,W,\tau} : \text{ColimProfile}_{i,W,\tau}(F_{\bullet}) \rightarrow \text{ApexProfile}_{i,W,\tau}(F_{\infty})$$

when Type IV diagnostics are invoked;

- (viii) monitored diagnostics;

- (ix) gate verdicts;

- (x) ledger policy and budget comparison

$$\Sigma\delta < \text{gap}_{\tau};$$

- (xi) artifact references;

- (xii) proof objects satisfying the Minimal Manifest Axiom;

- (xiii) formal or reconstructible checks of required claims.

Theorem 1.22 (Proposer–verifier separation). *Core pass/reject is determined only by verifier-side data. Proposer-side data may guide, motivate, or prioritize verification, but they do not determine the verdict.*

Proof. By Chapter 6 and Appendix G, Core verdicts are determined by repaired evaluation records, admissible representative and eligibility records when invoked, declared tower comparison artifacts when invoked, gate verdicts, monitored diagnostics, ledger comparison, and audit-readable manifest data. Proposer-side data are not part of this verifier-side rule. Therefore proposer-side data cannot determine Core pass/reject. $\square$

Remark 1.23 (The central safety boundary). This theorem is the central safety boundary of Part V. Search may be aggressive. Agents may be creative. The verifier remains conservative.

U.5. AI confidence, ranking, and plausibility

Definition 1.24 (AI confidence value). An *AI confidence value* is any scalar, categorical, probabilistic, linguistic, or model-generated estimate of plausibility, correctness, priority, or relevance associated with an AI-generated output.

Definition 1.25 (Agent ranking). An *agent ranking* is an ordering of candidate objects, regions, lemmas, proof routes, bridge statements, or artifacts produced by an agent.

Definition 1.26 (Plausibility score). A *plausibility score* is any non-verifier-side score estimating whether a candidate is worth checking. It may be human-assigned, AI-assigned, tool-assisted, or platform-derived.

Proposition 1.27 (AI confidence is not proof). *An AI confidence value is not a proof, not a Core gate verdict, not a monitored diagnostic, and not a budget comparison.*

Proof. A proof or Core verdict must be reconstructible through declared verifier-side conditions. An AI confidence value is a model-side estimate. It does not establish B-Gate⁺, repaired evaluation vanishing, eligibility, Type IV cleanliness, tower artifact validity, or ledger admissibility. $\square$

Proposition 1.28 (Ranking does not imply validity). *The rank assigned to a candidate by an agent does not imply that the candidate is valid.*

Proof. Ranking is a search or prioritization operation. Validity requires verification. A highly ranked candidate may fail a gate, diagnostic, ledger, eligibility, artifact, or bridge condition. $\square$

Proposition 1.29 (Plausibility does not imply verifiability). *A plausible candidate is not necessarily verifiable.*

Proof. Plausibility concerns apparent promise. Verifiability requires typed objects, declared criteria, artifact support, and reconstructible checks. A candidate may appear plausible while lacking the data needed for verification. $\square$

Remark 1.30 (Correct use of confidence). AI confidence, rankings, and plausibility scores may be useful for triage. They may determine what is checked first. They may not determine what is accepted.

U.6. Agent outputs and status labels

Definition 1.31 (Agent output). An *agent output* is any object produced by an agent, including text, code, theorem statements, diagrams, proof sketches, formal fragments, data files, certificate candidates, maps, rankings, annotations, logs, or status updates.

Definition 1.32 (Agent-output status label). Every agent output used in the AK workflow must be assigned one of the following status labels:

- (i) *search artifact*;
- (ii) *candidate*;
- (iii) *draft*;
- (iv) *advisory*;
- (v) *operational policy*;

- (vi) *interface specification*;
- (vii) *[Spec]*;
- (viii) *formal fragment*;
- (ix) *verified formal fragment*;
- (x) *proof sketch*;
- (xi) *bridge draft*;
- (xii) *bridge theorem*;
- (xiii) *Core certificate candidate*;
- (xiv) *Core verified*;
- (xv) *audit-ready*;
- (xvi) *non-claim*;
- (xvii) *rejected*;
- (xviii) *incomplete*;
- (xix) *deprecated*;
- (xx) *superseded*.

Definition 1.33 (Unlabeled agent output). An *unlabeled agent output* is an agent output lacking an explicit status label.

Definition 1.34 (Status promotion). *Status promotion* is an operation that moves an output from a lower-status class to a higher-status class, for example:

candidate $\longrightarrow$ Core certificate candidate $\longrightarrow$ Core verified.

Definition 1.35 (Status demotion). *Status demotion* is an operation that moves an output to a lower or safer status class, for example:

Core certificate candidate $\longrightarrow$ incomplete or verified formal fragment $\longrightarrow$ deprecated.

Proposition 1.36 (Unlabeled outputs are non-verdict by default). *An unlabeled agent output is non-verdict data by default.*

Proof. Without a status label, an auditor cannot determine whether the output is intended as a search artifact, theorem, proof fragment, policy, bridge, certificate, or non-claim. Therefore it cannot affect Core verdicts. $\square$

Proposition 1.37 (Promotion requires evidence). *An agent output may be promoted to a higher status only if the required evidence for that higher status is supplied.*

Proof. Higher status labels impose stronger semantic requirements. For example, Core verified status requires verifier-side data, and bridge theorem status requires a proved bridge. Without such evidence, the promotion is unsupported. $\square$

Remark 1.38 (Labeling prevents claim inflation). Status labels prevent a search artifact from being misread as a theorem, or a draft from being misread as a verified proof.

U.7. Agent action semantics

Definition 1.39 (Generate). An agent action *generate* produces a new candidate object or artifact. Generation does not imply verification.

Definition 1.40 (Transform). An agent action *transform* modifies an existing object into another form, such as translating, normalizing, refactoring, formalizing, compressing, expanding, or summarizing it. Transformation does not preserve validity unless preservation is proved or checked.

Definition 1.41 (Rank). An agent action *rank* orders candidates by a declared priority function, heuristic, score, or policy. Ranking does not imply proof.

Definition 1.42 (Annotate). An agent action *annotate* adds metadata, comments, explanations, warnings, or status labels to an object. Annotation does not change the mathematical content unless explicitly declared.

Definition 1.43 (Verify). An agent action *verify* checks a claim, object, formal fragment, bridge, or certificate against declared verifier-side criteria. Verification must specify the criteria being checked.

Definition 1.44 (Audit). An agent action *audit* checks whether the verification record, artifacts, manifest, provenance, replay data, and proof object schema allow independent reconstruction.

Definition 1.45 (Replay). An agent action *replay* re-executes, recomputes, or reconstructs a declared workflow from stored artifacts and manifests. Replay checks reproducibility, not necessarily mathematical truth beyond the replayed criteria.

Proposition 1.46 (Action semantics are non-interchangeable). *Generation, transformation, ranking, annotation, verification, audit, and replay are distinct actions and cannot be substituted for one another without explicit justification.*

Proof. Each action has a different semantic effect. Generation creates candidates. Ranking prioritizes. Verification checks validity. Audit checks reconstructibility. Replay checks reproducibility. Since their outputs answer different questions, they cannot be interchanged. $\square$

Remark 1.47 (Transformation caution). A transformed proof, transformed definition, or transformed certificate may no longer be equivalent to the source object. Equivalence requires a preservation check or proof.

U.8. Verification authority

Definition 1.48 (Verification authority). A *verification authority* is a declared procedure or agent whose output is allowed to establish a specific verifier-side status, such as verified formal fragment, passed gate, passed diagnostic, passed budget check, passed manifest check, or replay success.

Definition 1.49 (Authority scope). The *authority scope* of a verification authority is the class of statements, objects, or checks it is authorized to verify.

Definition 1.50 (Authority environment). The *authority environment* is the declared versioned environment in which a verification authority operates, including tool versions, libraries, assumptions, configuration, input artifacts, and accepted axioms.

Proposition 1.51 (Verification authority is scoped). *A verification authority verifies only claims within its declared authority scope and declared environment.*

Proof. Different tools and agents check different properties. A type checker may verify type correctness but not mathematical soundness of an informal bridge. A persistence engine may compute barcodes but not prove a PDE regularity theorem. A proof assistant verifies only relative to its formal environment. Thus authority must be scoped and environment-relative. $\square$

Remark 1.52 (Tool output requires interpretation). Even deterministic tool output requires interpretation. A successful run proves only what the tool is specified, configured, and authorized to check.

U.9. Human review

Definition 1.53 (Human review). *Human review* is the process by which a human agent reads, checks, critiques, edits, approves, rejects, or interprets an AK-related object.

Definition 1.54 (Human approval). *Human approval* is an explicit statement that a human agent accepts an object for a declared purpose.

Definition 1.55 (Human mathematical judgment). *Human mathematical judgment* is a human assessment of correctness, plausibility, rigor, usefulness, or risk. It may guide verification, but it is not identical to verifier-side proof unless supported by proof data.

Proposition 1.56 (Human approval is scoped). *Human approval establishes only the approved status within the declared scope. It does not automatically establish Core validity.*

Proof. A human may approve style, structure, plausibility, publication readiness, or readiness for verification. Core validity requires Core verifier-side checks. Therefore human approval is scoped to its declared purpose. $\square$

Remark 1.57 (Human review remains important). Human review is crucial for detecting conceptual gaps, claim inflation, poor definitions, missing hypotheses, and misaligned interfaces. However, review and verification should remain semantically distinct.

U.10. Agent provenance

Definition 1.58 (Agent provenance). *Agent provenance* is metadata recording which agents produced, transformed, verified, audited, approved, rejected, deprecated, or superseded an object.

Definition 1.59 (Provenance record). A *provenance record* contains:

- (i) the agent identifier or class;
- (ii) the agent role;
- (iii) the action performed;
- (iv) the input object;
- (v) the output object;
- (vi) timestamp or version reference, if used;
- (vii) authority scope, if verification is claimed;
- (viii) status label before and after the action;

- (ix) artifact references;
- (x) audit notes.

Definition 1.60 (Versioned provenance). *Versioned provenance* is provenance tied to immutable or version-controlled artifact identifiers.

Proposition 1.61 (Provenance supports audit but does not prove validity). *Agent provenance supports auditability, but it does not by itself prove mathematical validity.*

Proof. Provenance records who did what. Validity requires that the resulting object satisfies the relevant mathematical or verifier-side conditions. The former supports reconstruction, but does not replace verification. $\square$

Remark 1.62 (Why provenance matters). Provenance is important for reproducibility, accountability, debugging, and version control. It also helps detect when an output has moved from search artifact to certificate candidate to verified object.

U.11. Agent failure modes

Definition 1.63 (Agent failure mode). An *agent failure mode* is a way in which an agent output may be misleading, invalid, incomplete, unsupported, stale, or misclassified.

Definition 1.64 (Hallucinated claim). A *hallucinated claim* is a claim generated by an agent without adequate support, proof, citation, artifact, or verification.

Definition 1.65 (Claim inflation). *Claim inflation* occurs when a lower-status output, such as a search artifact, advisory indicator, draft, confidence score, or [Spec] bridge, is described or used as if it were a theorem or verified certificate.

Definition 1.66 (Scope overrun). *Scope overrun* occurs when an agent or tool output is used beyond its declared authority scope.

Definition 1.67 (Artifact drift). *Artifact drift* occurs when an artifact changes, is replaced, becomes stale, or becomes inconsistent with the certificate or proof object that references it.

Definition 1.68 (Bridge inflation). *Bridge inflation* occurs when a bridge draft, bridge program, or [Spec] bridge is treated as a proved bridge theorem.

Definition 1.69 (Formalization mismatch). *Formalization mismatch* occurs when a verified formal statement is interpreted as proving a stronger or different informal mathematical claim.

Proposition 1.70 (Agent failure modes require status downgrade). *If an agent failure mode is detected in an object, the object must be downgraded to an appropriate non-verified status until repaired and rechecked.*

Proof. A detected failure mode undermines the current status of the object. Until the defect is repaired and verification or audit is repeated, the previous verified or candidate status is not justified. $\square$

Remark 1.71 (Failure modes are expected). Agent failure modes are not exceptional in a search-heavy workflow. The platform must assume they will occur and must provide status labels, provenance, and audit mechanisms to contain them.

U.12. Agent outputs and Core claims

Definition 1.72 (Core claim by agent). A *Core claim by agent* is any assertion by an agent that a Core condition, theorem, gate, diagnostic, budget comparison, representative condition, eligibility condition, artifact condition, or certificate status holds.

Definition 1.73 (Admissible agent Core claim). An agent Core claim is *admissible* only if it is supported by verifier-side data sufficient to reconstruct the claimed Core status.

Proposition 1.74 (Agent assertion does not establish Core claim). *An agent assertion that a Core claim holds does not establish that claim unless the supporting verifier-side data are present.*

Proof. Core claims require verification. An assertion is a statement about the claim, not the verification itself. Without verifier-side data, the assertion is unsupported. $\square$

Remark 1.75 (This includes AI and human assertions). This rule applies equally to AI agents and human agents. Neither confidence nor authority substitutes for the certificate record.

U.13. Agent interaction with proof assistants

Definition 1.76 (Proof assistant interaction). A *proof assistant interaction* is an action in which an agent submits, modifies, checks, or reads formal code or proof objects in a proof assistant.

Definition 1.77 (Formal fragment). A *formal fragment* is a piece of formal code, theorem statement, definition, lemma, tactic script, or proof script intended for proof assistant checking.

Definition 1.78 (Verified formal fragment). A *verified formal fragment* is a formal fragment that has been successfully checked by the declared proof assistant under the declared environment.

Definition 1.79 (Formal interpretation theorem). A *formal interpretation theorem* is a theorem connecting a verified formal statement to an intended informal mathematical statement or interface claim.

Proposition 1.80 (Verified formal fragment is scoped to its formal statement). *A verified formal fragment establishes only the formal statement that was checked in the declared environment.*

Proof. A proof assistant checks formal syntax, typing, and proof obligations relative to its environment. It does not automatically verify informal interpretations, external mathematical claims, or problem-specific meanings not encoded in the formal statement. $\square$

Proposition 1.81 (Formal interpretation requires a bridge). *A verified formal fragment may support an informal or problem-specific claim only if an interpretation theorem connects the formal statement to that claim.*

Proof. The formal statement and the intended informal claim may differ in hypotheses, semantics, or scope. A connecting theorem is required to justify the interpretation. $\square$

Remark 1.82 (Formalization boundary). A formal proof of a formal statement may still require an interpretation theorem connecting that statement to the intended mathematical claim. This is especially important for problem interfaces.

U.14. Agent manifest entries

Definition 1.83 (Agent manifest entry). An *agent manifest entry* records:

- (i) the agent identity or class;
- (ii) the agent role;
- (iii) the action performed;
- (iv) the input artifacts;
- (v) the output artifacts;
- (vi) the output status label;
- (vii) any confidence, score, plausibility, or ranking, if present;
- (viii) the authority scope, if verification is claimed;
- (ix) the authority environment, if a tool or proof assistant is used;
- (x) artifact references;
- (xi) audit notes.

Definition 1.84 (Agent action log). An *agent action log* is an ordered list of agent manifest entries recording the lifecycle of a search artifact, proof object, certificate candidate, formal fragment, bridge statement, or verified object.

Definition 1.85 (Agent lifecycle). The *agent lifecycle* of an object is the sequence:

generated $\longrightarrow$ transformed $\longrightarrow$ labeled $\longrightarrow$ checked $\longrightarrow$ audited $\longrightarrow$ curated,

with stages omitted or repeated as appropriate.

Proposition 1.86 (Agent manifest entries support reproducibility). *Agent manifest entries support reproducibility by recording how objects were generated, transformed, verified, audited, and curated.*

Proof. Reproducibility requires reconstruction of the workflow. Agent manifest entries record the relevant actions, inputs, outputs, roles, status labels, environments, and artifact references. Thus they support reconstruction. $\square$

Remark 1.87 (Manifest is not verdict). An agent manifest entry records actions and status. It does not itself establish mathematical truth unless it contains or references the required verifier-side evidence.

U.15. Agent interaction with bridges and proof objects

Definition 1.88 (Agent-generated bridge draft). An *agent-generated bridge draft* is a proposed implication connecting two layers of the AK framework, such as:

Core-visible condition $\implies$ problem-specific conclusion,

generated by an agent before proof.

Definition 1.89 (Agent-generated proof object). An *agent-generated proof object* is a structured proof object or proof-object candidate generated, filled, transformed, or assembled by an agent.

Definition 1.90 (Bridge verification requirement). A bridge draft may be promoted to a bridge theorem only after:

- (i) the statement is precisely formulated;
- (ii) all hypotheses are declared;
- (iii) all source and target semantics are fixed;
- (iv) the proof is supplied;
- (v) artifacts or formal fragments are attached where required;
- (vi) the bridge is checked by a declared verification authority.

Proposition 1.91 (Agent-generated bridge draft is not bridge theorem). *An agent-generated bridge draft is not a bridge theorem.*

Proof. A bridge theorem requires a precise statement, declared hypotheses, semantic compatibility, and proof. A draft supplies only a candidate bridge. Therefore it is not theorem-level until verified. $\square$

Proposition 1.92 (Agent-generated proof object is not proof by default). *An agent-generated proof object is not a proof by default.*

Proof. A proof object becomes proof-level only after its required fields, claims, formal fragments, bridges, artifacts, and verifier-side checks are supplied and verified. Generation alone does not establish those conditions. $\square$

Remark 1.93 (Bridge drafts are useful). Bridge drafts are useful because they expose what must be proved. Their value lies in structuring the proof search, not in establishing theorem-level validity.

U.16. Agent interaction with Core repaired objects

Definition 1.94 (Agent-produced repaired evaluation record). An *agent-produced repaired evaluation record* is an output in which an agent claims to have produced or computed:

$$\text{Eval}_{i,W,\tau}(F) = T_{\tau}^{\text{bar}}(W_W \mathbf{P}_i(F)).$$

Definition 1.95 (Agent-produced representative record). An *agent-produced representative record* is an output in which an agent claims to have produced or checked an admissible representative:

$$C_{\tau,W}F.$$

Definition 1.96 (Agent-produced tower artifact). An *agent-produced tower artifact* is an output in which an agent claims to have produced or checked:

$$\phi_{i,W,\tau} : \text{ColimProfile}_{i,W,\tau}(F_{\bullet}) \rightarrow \text{ApexProfile}_{i,W,\tau}(F_{\infty}).$$

Proposition 1.97 (Agent-produced Core records require verification). *Agent-produced repaired evaluation records, representative records, and tower artifacts require verification before they may support Core certification.*

Proof. These records are verifier-side data only when they are well-typed, artifact-supported, and checked under the declared Core protocol. Agent production alone does not establish those conditions. $\square$

Remark 1.98 (Core object sensitivity). Small changes in windows, thresholds, barcode policies, representative choices, or tower artifacts may change the verdict. Therefore agent-produced Core records must be treated as candidate records until verified.

U.17. Summary of agent semantics

Theorem 1.99 (Appendix U agent-semantics package). *Within the Search / Platform layer, the following package is available.*

- (i) *Agents may be human, AI, tool-based, platform-based, or hybrid.*
- (ii) *Agents may act as proposers, searchers, mappers, lifters, verifiers, auditors, curators, or adjudicators.*
- (iii) *Agent type does not determine validity.*
- (iv) *Proposer-side data do not determine Core verdicts.*
- (v) *Core pass/reject is determined only by verifier-side data.*
- (vi) *Verifier-side data include repaired evaluation records, admissible representatives when invoked, eligibility records when invoked, tower comparison artifacts when invoked, diagnostics, gates, ledgers, and artifact references.*
- (vii) *AI confidence is not proof.*
- (viii) *Agent rankings and plausibility scores do not imply validity.*
- (ix) *Agent outputs require explicit status labels.*
- (x) *Unlabeled outputs are non-verdict by default.*
- (xi) *Status promotion requires evidence.*
- (xii) *Agent actions such as generation, ranking, verification, audit, and replay are semantically distinct.*
- (xiii) *Verification authority is scoped and environment-relative.*
- (xiv) *Human approval is scoped.*
- (xv) *Provenance supports audit but does not prove validity.*
- (xvi) *Agent failure modes require status downgrade until repaired.*
- (xvii) *Agent assertions do not establish Core claims without verifier-side support.*
- (xviii) *Verified formal fragments are scoped to their checked formal statements.*
- (xix) *Formal interpretation requires a bridge.*
- (xx) *Agent manifest entries support reproducibility but do not themselves determine mathematical truth.*
- (xxi) *Agent-generated bridge drafts are not bridge theorem-level by default.*

(xxii) *Agent-generated proof objects are not proof by default.*

(xxiii) *Agent-produced Core records require verification before they may support Core certification.*

Proof. Items (i)–(iii) follow from the definitions of agent types and roles and Remark 1.10. Items (iv)–(vi) are Theorem 1.22 and the definition of verifier-side data. Items (vii)–(viii) are Propositions 1.27, 1.28, and 1.29. Items (ix)–(xi) follow from the status-label definitions and Propositions 1.36 and 1.37. Item (xii) is Proposition 1.46. Item (xiii) is Proposition 1.51. Item (xiv) is Proposition 1.56. Item (xv) is Proposition 1.61. Item (xvi) is Proposition 1.70. Item (xvii) is Proposition 1.74. Items (xviii)–(xix) are Propositions 1.80 and 1.81. Item (xx) follows from Proposition 1.86 and Remark 1.87. Items (xxi)–(xxii) are Propositions 1.91 and 1.92. Item (xxiii) is Proposition 1.97. $\square$

U.18. Explicit exclusions

The following are not asserted in Appendix U.

- No AI output is treated as proof by itself.
- No human assertion is treated as proof by itself.
- No agent confidence value is treated as a Core verdict.
- No agent ranking is treated as validity.
- No plausibility score is treated as verifiability.
- No search artifact is promoted to theorem status by authorship.
- No bridge draft is treated as a bridge theorem.
- No proof object generated by an agent is treated as proof by generation alone.
- No tool output is used beyond its declared authority scope.
- No proof assistant result is interpreted beyond the checked formal statement without an interpretation theorem.
- No unlabeled output affects Core pass/reject.
- No provenance record proves mathematical validity by itself.
- No platform storage event certifies an object.
- No agent approval replaces B-Gate⁺.
- No agent approval replaces the repaired topological condition

$$\text{Eval}_{1,w,\tau}(F) = 0.$$

- No agent approval replaces admissible representative records

$$C_{\tau,w}F.$$

- No agent approval replaces tower comparison artifacts

$$\phi_{i,w,\tau}.$$

- No agent approval replaces monitored Type IV diagnostics.
- No agent approval replaces the budget condition

$$\Sigma\delta < \text{gap}_{\tau}.$$

- No agent-generated validity map is treated as a theorem.
- No agent-produced repaired evaluation record is accepted without verification.
- No agent-produced tower artifact is accepted without verification.
- No status promotion is accepted without evidence.
- No artifact drift is ignored when detected.

U.19. Provenance and status

The material in this appendix depends on:

- (i) the claim taxonomy and Minimal Manifest discipline of Chapter 1;
- (ii) the repaired evaluation discipline of Appendix A;
- (iii) the admissible representative discipline of Appendix B;
- (iv) the eligible realization regime of Appendix C;
- (v) the monitored Type IV diagnostic and tower artifact discipline of Appendix D;
- (vi) the Core sovereignty doctrine of Chapter 6;
- (vii) the non-claim discipline of Chapter 7;
- (viii) the interface protocol of Chapter 8;
- (ix) the manifest schema of Appendix G;
- (x) the advisory-invariant discipline of Appendix H;
- (xi) the ledger catalog of Appendix M;
- (xii) the proof-first program discipline of Appendix NS-C.

Its role is to define how agents may participate in search and proof-management workflows without contaminating Core verdicts.

Status labels for Appendix U. *Search / Platform definitions:* Definitions 1.4, 1.5, 1.6, 1.7, 1.8, 1.9, 1.11, 1.12, 1.13, 1.14, 1.15, 1.16, 1.17, 1.18, 1.20, 1.21, 1.24, 1.25, 1.26, 1.31, 1.32, 1.33, 1.34, 1.35, 1.39, 1.40, 1.41, 1.42, 1.43, 1.44, 1.45, 1.48, 1.49, 1.50, 1.53, 1.54, 1.55, 1.58, 1.59, 1.60, 1.63, 1.64, 1.65, 1.66, 1.67, 1.68, 1.69, 1.72, 1.73, 1.76, 1.77, 1.78, 1.79, 1.83, 1.84, 1.85, 1.88, 1.89, 1.90, 1.94, 1.95, 1.96.

Search / Platform propositions/theorems: Theorems 1.22, 1.99, Propositions 1.27, 1.28, 1.29, 1.36, 1.37, 1.46, 1.51, 1.56, 1.61, 1.70, 1.74, 1.80, 1.81, 1.86, 1.91, 1.92, 1.97.

Operational cautions: Remarks 1.1, 1.2, 1.3, 1.10, 1.19, 1.23, 1.30, 1.38, 1.47, 1.52, 1.57, 1.62, 1.71, 1.75, 1.82, 1.87, 1.93, 1.98.

Explicit exclusions: Section U.18.

2 Appendix V: Hunter / Mapper / Lifter Search Protocols

V.1. Scope and search status

This appendix defines the Hunter / Mapper / Lifter search protocols used in the AK framework. It belongs to the Search / Platform Appendices. It does not enlarge the Core mathematical package of Chapters 1–8.

The central rule of this appendix is:

Search protocols generate candidates; they do not certify them.

In particular, no search hit, mapped structure, lifted candidate, priority score, agent recommendation, search map edge, or bridge draft may replace:

$$B\text{-Gate}^+, \quad \text{Eval}_{1,W,\tau}(F) = 0, \quad C_{\tau,W}F, \quad \phi_{i,W,\tau}, \quad (\mu_{\text{Collapse}}, u_{\text{Collapse}}) = (0, 0), \quad \Sigma\delta < \text{gap}_{\tau}.$$

Remark 2.1 (Search / Platform status). The protocols in this appendix organize discovery, mapping, prioritization, and candidate preparation. They do not establish mathematical validity by themselves.

Remark 2.2 (Relation to Appendix U). Appendix U defined agent semantics. The present appendix specializes those semantics to three search roles:

Hunter, Mapper, Lifter.

These roles may be performed by human agents, AI agents, tool agents, platform agents, or hybrid agents.

Remark 2.3 (Relation to Core re-entry). The purpose of the Hunter / Mapper / Lifter workflow is not to prove claims directly. Its purpose is to prepare candidates that may later be submitted to verifier-side Core re-entry. The re-entry boundary is therefore the semantic boundary between search and certification.

V.2. Search objects

Definition 2.4 (Search object). A *search object* is any object produced, selected, transformed, ranked, mapped, or lifted during search. Examples include:

- (i) candidate windows;
- (ii) candidate thresholds;
- (iii) candidate stable bands;

- (iv) candidate bridge statements;
- (v) candidate proof routes;
- (vi) advisory indicators;
- (vii) persistence profiles;
- (viii) candidate filtered objects;
- (ix) candidate tower diagrams;
- (x) candidate normalization routes;
- (xi) candidate problem-interface realizations;
- (xii) candidate repaired evaluation records;
- (xiii) candidate admissible representative records;
- (xiv) candidate tower comparison artifacts;
- (xv) draft certificate records;
- (xvi) proof object fragments.

Definition 2.5 (Search artifact). A *search artifact* is a stored or referenced output of a search process. It may include text, code, diagrams, rankings, maps, datasets, proof sketches, logs, generated candidates, or structured records.

Definition 2.6 (Candidate). A *candidate* is a search object that has been selected for possible further development, mapping, lifting, verification, formalization, or audit.

Definition 2.7 (Candidate status). A candidate has one of the following statuses:

- (i) *raw*;
- (ii) *ranked*;
- (iii) *mapped*;
- (iv) *lift-attempted*;
- (v) *lifted*;
- (vi) *verification-ready*;
- (vii) *blocked*;
- (viii) *rejected*;
- (ix) *incomplete*;
- (x) *deprecated*;
- (xi) *superseded*.

Proposition 2.8 (Search artifacts are non-verdict by default). *A search artifact does not affect Core pass/reject unless it is converted into verifier-side data and checked under the Core protocol.*

Proof. Search artifacts are proposer-side data. By Appendix U, proposer-side data do not determine Core verdicts. Only verifier-side data determine Core pass/reject. $\square$

Remark 2.9 (Search artifact value). Search artifacts may still be highly valuable. They support discovery, comparison, reproducibility, prioritization, debugging, and proof planning. Their limitation is logical, not operational.

V.3. Hunter protocol

Definition 2.10 (Hunter). A *Hunter* is an agent role whose task is to find candidate objects, regions, lemmas, bridges, failure modes, proof routes, or artifact references in a declared search space.

Definition 2.11 (Hunt space). A *hunt space* is a declared space of possible search targets. Examples include:

- (i) window families;
- (ii) threshold bands;
- (iii) tower configurations;
- (iv) arithmetic interface routes;
- (v) PDE observable families;
- (vi) normalization choices;
- (vii) bridge lemma templates;
- (viii) candidate proof programs;
- (ix) formalization fragments;
- (x) certificate DAG fragments;
- (xi) known failure-mode neighborhoods.

Definition 2.12 (Hunt objective). A *hunt objective* is the declared goal of a Hunter protocol, such as finding a stable band, locating a bridge candidate, identifying a Type IV proxy, searching for a counterexample, or discovering a Core-readable realization route.

Definition 2.13 (Hunt output). A *hunt output* is a list, set, graph, table, or ranked family of candidates produced by a Hunter, together with any scores, rankings, reasons, constraints, or artifact references.

Definition 2.14 (Hunter protocol). A *Hunter protocol* specifies:

- (i) the hunt space;
- (ii) the hunt objective;
- (iii) the search constraints;
- (iv) the candidate selection rule;

- (v) the ranking or scoring method, if any;
- (vi) stopping or continuation conditions, if any;
- (vii) the output status labels;
- (viii) artifact references.

Proposition 2.15 (Hunter output is not certificate). *A Hunter output is not a Core certificate.*

Proof. Hunter output is a candidate-generating search artifact. A Core certificate requires verifier-side data satisfying the Minimal Manifest Axiom, repaired evaluation records, gates, diagnostics, ledgers, tower artifacts when invoked, and artifact requirements. Thus Hunter output is not a certificate. $\square$

Remark 2.16 (Hunter success). Hunter success means that promising candidates were found. It does not mean that the candidates are valid.

V.4. Mapper protocol

Definition 2.17 (Mapper). A *Mapper* is an agent role whose task is to organize search objects into maps, dependency structures, diagrams, classifications, validity landscapes, certificate DAG fragments, or bridge-route graphs.

Definition 2.18 (Search map). A *search map* is a structured representation of relations among search objects. It may include:

- (i) dependency arrows;
- (ii) obstruction labels;
- (iii) candidate bridge routes;
- (iv) window overlap relations;
- (v) tower comparison relations;
- (vi) status labels;
- (vii) search priorities;
- (viii) known gaps;
- (ix) failure-mode labels;
- (x) artifact references.

Definition 2.19 (Map edge semantics). A *map edge semantics* specifies what an edge in a search map means. Allowed edge types include:

- (i) dependency;
- (ii) analogy;
- (iii) candidate implication;
- (iv) verified implication;

- (v) obstruction;
- (vi) refinement;
- (vii) replacement;
- (viii) contradiction;
- (ix) artifact reference;
- (x) status transition.

Definition 2.20 (Mapper output). A *Mapper output* is a search map, validity map fragment, dependency diagram, certificate DAG fragment, bridge graph, obstruction map, or classification table produced by a Mapper.

Definition 2.21 (Mapper protocol). A *Mapper protocol* specifies:

- (i) the objects to be mapped;
- (ii) the relation types;
- (iii) the edge semantics;
- (iv) the status labels;
- (v) the artifact references;
- (vi) the update policy;
- (vii) the conflict-resolution policy.

Proposition 2.22 (Mapper output is navigation, not theorem). *A Mapper output is a navigation artifact and does not establish theorems by itself.*

Proof. A map records relations, candidates, priorities, dependencies, or gaps. The truth of a theorem requires proof or verification of the relevant statements. A map may point to such proof objects, but it does not replace them. □

Remark 2.23 (Map edges must be typed). Edges in a search map must be typed. An edge may mean dependency, analogy, conjectural implication, verified implication, blocked route, or artifact reference. Untyped edges invite claim inflation.

V.5. Lifter protocol

Definition 2.24 (Lifter). A *Lifter* is an agent role whose task is to transform a search object into a Core-readable object candidate, bridge candidate, certificate candidate, formal fragment candidate, or proof object candidate.

Definition 2.25 (Lift input). A *lift input* is a candidate produced by a Hunter or organized by a Mapper and selected for possible conversion into verifier-side or verifier-adjacent data.

Definition 2.26 (Lift target). A *lift target* is the declared type into which the candidate is intended to be transformed. Examples include:

$$\text{FiltCh}((k), \quad \text{Pers}_k^{\text{cons}}, \quad \text{Eval}_{i,W,\tau}(F), \quad C_{\tau,W}F, \quad \phi_{i,W,\tau},$$

a ledger entry, a bridge lemma candidate, a formal fragment, or a certificate record candidate.

Definition 2.27 (Lift output). A *lift output* is one of:

- (i) a Core-readable filtered object candidate;
- (ii) a constructible persistence object candidate;
- (iii) a repaired evaluation record candidate
 $\text{Eval}_{i,W,\tau}(F);$
- (iv) an admissible representative candidate
 $C_{\tau,W}F;$
- (v) an eligible realized object candidate;
- (vi) a tower comparison artifact candidate
 $\phi_{i,W,\tau};$
- (vii) a tower diagnostic candidate;
- (viii) a ledger entry candidate;
- (ix) a bridge lemma candidate;
- (x) a certificate record candidate;
- (xi) a formal fragment candidate.

Definition 2.28 (Lifter protocol). A *Lifter protocol* specifies:

- (i) the lift input;
- (ii) the lift target;
- (iii) the transformation rule;
- (iv) the required declarations;
- (v) constructibility requirements;
- (vi) representative and eligibility requirements, if invoked;
- (vii) tower artifact requirements, if Type IV diagnostics are invoked;
- (viii) error or defect accounting;
- (ix) status labels before and after lifting;
- (x) artifact references.

Proposition 2.29 (Lifted candidate is not verified by lifting). *A lifted candidate is not verified merely because it has been lifted.*

Proof. Lifting changes the type or representation of a candidate. Verification checks whether the resulting object satisfies declared verifier-side conditions. These are distinct actions by Appendix U. $\square$

Proposition 2.30 (Core-looking lift is not Core-verified). *A lift output that syntactically resembles a Core object, such as*

$$\text{Eval}_{i,W,\tau}(F), \quad C_{\tau,W}F, \quad \phi_{i,W,\tau},$$

is not Core-verified until checked under the declared Core protocol.

Proof. A Core-looking object may be malformed, unsupported, nonconstructible, incorrectly parameterized, or inconsistent with artifacts. Core verification requires typed declarations, artifact support, and verifier-side checks. Therefore syntactic form alone is insufficient. $\square$

Remark 2.31 (Lifting is the re-entry preparation step). Lifting is the natural preparation step before Core re-entry. It attempts to turn search artifacts into objects that the Core verifier can actually inspect.

V.6. Search lifecycle

Definition 2.32 (Search lifecycle). The *search lifecycle* is the ordered process:

$$\text{hunt} \longrightarrow \text{map} \longrightarrow \text{lift} \longrightarrow \text{verify} \longrightarrow \text{audit}.$$

Definition 2.33 (Lifecycle status). A search object may have one of the following lifecycle statuses:

- (i) *found*;
- (ii) *mapped*;
- (iii) *prioritized*;
- (iv) *lift-attempted*;
- (v) *lifted*;
- (vi) *verification-ready*;
- (vii) *verified*;
- (viii) *audited*;
- (ix) *rejected*;
- (x) *blocked*;
- (xi) *incomplete*;
- (xii) *deprecated*;
- (xiii) *superseded*.

Definition 2.34 (Lifecycle transition). A *lifecycle transition* is a status change in a search object. Every transition must record the agent, action, evidence, and artifact references supporting the transition.

Proposition 2.35 (Lifecycle advancement is not validity monotonicity). *Advancing through the search lifecycle does not monotonically increase mathematical validity.*

Proof. A candidate may be found, mapped, and lifted, yet fail verification. Lifecycle stages record workflow progress, not mathematical truth. Validity is established only at the appropriate verification stage. $\square$

Remark 2.36 (Why lifecycle status matters). Lifecycle status helps manage large search spaces. It prevents premature conversion of found or mapped objects into verified claims.

V.7. Search scores

Definition 2.37 (Search score). A *search score* is a scalar, vector, categorical, or ordered value assigned to a search object for prioritization. Examples include:

- (i) plausibility score;
- (ii) structural alignment score;
- (iii) advisory stability score;
- (iv) bridge-likelihood score;
- (v) defect-risk score;
- (vi) novelty score;
- (vii) proof-readiness score;
- (viii) agent confidence value;
- (ix) re-entry-readiness score;
- (x) failure-risk score.

Definition 2.38 (Score semantics). A *score semantics* specifies what a search score measures, how it is computed, and what it does not measure.

Definition 2.39 (Proof-readiness score). A *proof-readiness score* estimates whether a candidate appears ready for formalization, verifier-side checking, or proof-object assembly. It does not establish proof.

Proposition 2.40 (Search score is not proof). *A search score is not proof and does not determine Core pass/reject.*

Proof. A search score is proposer-side prioritization data. By Appendix U, proposer-side data do not determine Core verdicts. $\square$

Remark 2.41 (Score discipline). Scores may be useful for selecting what to inspect. They should not be used to accept or reject mathematical claims without verification.

V.8. Search-to-Core re-entry

Definition 2.42 (Search-to-Core re-entry). *Search-to-Core re-entry* is the process by which a lifted candidate is submitted to the Core verifier. It requires:

- (i) declared objects;
- (ii) declared windows W ;
- (iii) threshold and collapse policies;
- (iv) repaired evaluation records

$$\text{Eval}_{i,W,\tau}(F) = T_{\tau}^{\text{bar}}(W_W \mathbf{P}_i(F));$$

- (v) admissible representative records

$$C_{\tau,W}F$$

when invoked;

- (vi) eligibility records when the Ext^1 -clause is invoked;
- (vii) declared tower comparison artifacts

$$\phi_{i,W,\tau}$$

when Type IV diagnostics are invoked;

- (viii) monitored diagnostics;
- (ix) gate verdicts;
- (x) ledger policy and budget comparison;
- (xi) artifact references;
- (xii) proof object or certificate record schema;
- (xiii) status labels for all imported search artifacts.

Definition 2.43 (Re-entry candidate). A *re-entry candidate* is a lifted object prepared for Core verification but not yet verified.

Definition 2.44 (Re-entry package). A *re-entry package* is the collection of artifacts, declarations, labels, records, and candidate verifier-side data submitted together for Core verification.

Definition 2.45 (Re-entry failure). A *re-entry failure* occurs when a lifted candidate cannot satisfy the required manifest, typing, constructibility, repaired evaluation, representative, eligibility, diagnostic, tower artifact, ledger, or gate preconditions needed for Core verification.

Proposition 2.46 (Re-entry candidate is not Core certificate). *A re-entry candidate is not a Core certificate until it passes the required verifier-side checks.*

Proof. A re-entry candidate is prepared for verification. A Core certificate has passed verification according to the declared Core conditions. Preparation is not verification. $\square$

Remark 2.47 (Re-entry is the semantic boundary). Before re-entry, data are search-side. After successful verification, data may become Core-certified. This boundary must remain explicit.

V.9. Hunter / Mapper / Lifter failure modes

Definition 2.48 (Hunter failure). A *Hunter failure* occurs when search produces candidates that are irrelevant, unsupported, misranked, duplicate, malformed, stale, outside the declared hunt space, or inconsistent with the hunt objective.

Definition 2.49 (Mapper failure). A *Mapper failure* occurs when relations, edges, dependencies, status labels, or map structures are incorrect, ambiguous, untyped, cyclic when acyclicity is required, inconsistent with artifacts, or semantically inflated.

Definition 2.50 (Lifter failure). A *Lifter failure* occurs when a candidate cannot be converted into the declared target type, fails constructibility, lacks representative compatibility, lacks eligibility, lacks required tower artifacts, has unledgered defects, or lacks required artifact references.

Definition 2.51 (Re-entry failure mode). A *re-entry failure mode* is a failure occurring after lifting but before Core verification, including missing manifest fields, mismatched labels, invalid artifacts, missing repaired evaluations, missing tower artifacts, or unsupported ledger entries.

Proposition 2.52 (Search failure requires status downgrade). *If a Hunter, Mapper, Lifter, or re-entry failure is detected, the affected object must be downgraded to an appropriate non-verified status until repaired and rechecked.*

Proof. A search failure undermines the reliability of the affected object or its workflow status. By Appendix U, detected failure modes require status downgrade until repair and rechecking. $\square$

Remark 2.53 (Failure mode localization). The distinction among Hunter, Mapper, Lifter, and re-entry failures helps locate the repair task: search objective, relation map, Core-readability conversion, or verifier-side submission.

V.10. Search records

Definition 2.54 (Search record). A *search record* is a structured record of a search process, including hunt, map, lift, verification, re-entry, and audit status.

Definition 2.55 (Hunter record). A *Hunter record* contains:

- (i) hunt space;
- (ii) hunt objective;
- (iii) search constraints;
- (iv) candidates found;
- (v) scores or rankings, if used;
- (vi) selection policy;
- (vii) rejected or blocked candidates, if tracked;
- (viii) artifact references.

Definition 2.56 (Mapper record). A *Mapper record* contains:

- (i) mapped objects;

- (ii) relation types;
- (iii) edge semantics;
- (iv) status labels;
- (v) blocked routes;
- (vi) unresolved gaps;
- (vii) conflict-resolution notes;
- (viii) artifact references.

Definition 2.57 (Lifter record). A *Lifter record* contains:

- (i) lift input;
- (ii) lift target;
- (iii) transformation rule;
- (iv) resulting lifted object;
- (v) constructibility status;
- (vi) representative and eligibility status, if invoked;
- (vii) tower artifact status, if invoked;
- (viii) defects and ledger entries;
- (ix) re-entry readiness status;
- (x) artifact references.

Definition 2.58 (Re-entry record). A *re-entry record* contains:

- (i) the re-entry candidate;
- (ii) the re-entry package;
- (iii) required verifier-side data;
- (iv) missing verifier-side data, if any;
- (v) Core verification status;
- (vi) failure modes, if any;
- (vii) artifact references.

Proposition 2.59 (Search records support reproducibility). *Search records support reproducibility by recording how candidates were found, mapped, lifted, and prepared for verification.*

Proof. Reproducibility requires reconstructing the workflow. Search records record the relevant inputs, outputs, actions, statuses, and artifact references. Therefore they support reconstruction. $\square$

Remark 2.60 (Search record is not proof record). A search record explains how a candidate was obtained. It does not by itself prove the candidate.

V.11. Search protocol manifest entries

Definition 2.61 (Search protocol manifest entry). A *search protocol manifest entry* records:

- (i) the protocol type: Hunter, Mapper, Lifter, re-entry, or hybrid;
- (ii) the agent identity or class;
- (iii) the search objective;
- (iv) input artifacts;
- (v) output artifacts;
- (vi) output status labels;
- (vii) scores, rankings, or confidence values, if present;
- (viii) edge semantics, if mapping is used;
- (ix) lift target, if lifting is used;
- (x) verifier-side target, if re-entry is attempted;
- (xi) failure modes, if detected;
- (xii) repair history, if any;
- (xiii) Core re-entry status.

Definition 2.62 (Hybrid search protocol). A *hybrid search protocol* combines Hunter, Mapper, Lifter, re-entry, verification, or audit actions in a single workflow.

Proposition 2.63 (Hybrid protocol does not collapse role distinctions). A *hybrid search protocol does not erase the semantic distinction among hunting, mapping, lifting, verification, re-entry, and audit*.

Proof. A hybrid workflow may combine actions operationally. However, each action has a distinct semantic role by Appendix U. Thus the distinctions must remain visible in the manifest. $\square$

V.12. Search protocol invariants

Definition 2.64 (Search protocol invariant). A *search protocol invariant* is a rule that must remain true throughout the Hunter / Mapper / Lifter workflow.

Definition 2.65 (Non-verdict invariant). The *non-verdict invariant* states that no search-side object determines Core pass/reject before verified Core re-entry.

Definition 2.66 (Typed-edge invariant). The *typed-edge invariant* states that every edge in a search map must have declared semantics.

Definition 2.67 (Lift-target invariant). The *lift-target invariant* states that every lift must declare its target type before the lifted output is used.

Definition 2.68 (Re-entry invariant). The *re-entry invariant* states that no lifted candidate may be treated as Core-certified until it passes the required verifier-side checks.

Proposition 2.69 (Search protocol invariants prevent claim inflation). *The non-verdict, typed-edge, lift-target, and re-entry invariants prevent search artifacts from being silently promoted into Core claims.*

Proof. The non-verdict invariant blocks search-side acceptance. The typed-edge invariant blocks ambiguous map implications. The lift-target invariant blocks untyped conversion. The re-entry invariant blocks treating prepared candidates as verified certificates. Together, these rules prevent claim inflation. $\square$

V.13. Summary of Hunter / Mapper / Lifter protocols

Theorem 2.70 (Appendix V search-protocol package). *Within the Search / Platform layer, the following package is available.*

- (i) *Search objects and search artifacts are non-verdict by default.*
- (ii) *Hunters generate candidates from declared hunt spaces.*
- (iii) *Hunter outputs are not Core certificates.*
- (iv) *Mappers organize candidates into maps, diagrams, dependencies, validity landscapes, or classifications.*
- (v) *Mapper outputs are navigation artifacts, not theorems.*
- (vi) *Map edges must have declared semantics.*
- (vii) *Lifters attempt to transform candidates into Core-readable objects, bridge candidates, certificate candidates, formal fragments, or proof-object candidates.*
- (viii) *Lift outputs may include candidates for*

$$\text{Eval}_{i,W,\tau}, \quad C_{\tau,W}F, \quad \phi_{i,W,\tau},$$
but these are not verified merely by lifting.
- (ix) *Core-looking lift outputs are not Core-verified until checked.*
- (x) *Search lifecycle status records workflow progress, not mathematical validity.*
- (xi) *Search scores are not proof.*
- (xii) *Search-to-Core re-entry requires verifier-side manifest data.*
- (xiii) *Re-entry candidates are not Core certificates until verified.*
- (xiv) *Hunter, Mapper, Lifter, and re-entry failures require status downgrade until repaired and rechecked.*
- (xv) *Search records support reproducibility but do not prove validity.*
- (xvi) *Hybrid search protocols must preserve semantic role distinctions.*
- (xvii) *Search protocol invariants prevent claim inflation.*

Proof. Item (i) is Proposition 2.8. Items (ii)–(iii) follow from the Hunter definitions and Proposition 2.15. Items (iv)–(vi) follow from the Mapper definitions, Proposition 2.22, and the typed-edge discipline. Items (vii)–(ix) follow from the Lifter definitions and Propositions 2.29 and 2.30. Item (x) is Proposition 2.35. Item (xi) is Proposition 2.40. Items (xii)–(xiii) follow from the Search-to-Core re-entry definitions and Proposition 2.46. Item (xiv) is Proposition 2.52. Item (xv) follows from Proposition 2.59 and Remark 2.60. Item (xvi) is Proposition 2.63. Item (xvii) is Proposition 2.69. $\square$

V.14. Explicit exclusions

The following are not asserted in Appendix V.

- No Hunter output is treated as proof.
- No Mapper output is treated as theorem.
- No Lifter output is treated as verified merely by lifting.
- No search score is treated as validity.
- No agent confidence value is treated as a Core verdict.
- No search artifact updates Core pass/reject.
- No search map edge is treated as implication unless typed and verified.
- No candidate stable band is treated as Core stable band without diagnostics.
- No candidate bridge is treated as bridge theorem.
- No candidate repaired evaluation record

$$\text{Eval}_{i,W,\tau}(F)$$

is treated as verified merely by being produced.

- No candidate admissible representative

$$C_{\tau,W}F$$

is treated as verified merely by being produced.

- No candidate tower artifact

$$\phi_{i,W,\tau}$$

is treated as verified merely by being produced.

- No re-entry candidate is treated as Core certificate before verification.
- No search record is treated as proof record.
- No hybrid search workflow collapses proposer and verifier roles.
- No search protocol replaces B-Gate⁺.
- No search protocol replaces the repaired topological condition

$$\text{Eval}_{1,W,\tau}(F) = 0.$$

- No search protocol replaces admissible representative records.
- No search protocol replaces monitored Type IV diagnostics.
- No search protocol replaces the tower comparison artifact

$$\phi_{i,W,\tau}.$$

- No search protocol replaces the budget condition

$$\Sigma\delta < \text{gap}_{\tau}.$$

V.15. Provenance and status

The material in this appendix depends on:

- (i) the Agent Semantics of Appendix U;
- (ii) the repaired evaluation discipline of Appendix A;
- (iii) the admissible representative discipline of Appendix B;
- (iv) the eligible realization regime of Appendix C;
- (v) the monitored Type IV diagnostic and tower artifact discipline of Appendix D;
- (vi) the Core sovereignty doctrine of Chapter 6;
- (vii) the non-claim discipline of Chapter 7;
- (viii) the interface protocol of Chapter 8;
- (ix) the manifest schema of Appendix G;
- (x) the advisory-invariant discipline of Appendix H;
- (xi) the ledger catalog of Appendix M.

Its role is to define how search processes may discover, organize, and prepare candidates without converting search into proof.

Status labels for Appendix V. *Search / Platform definitions:* Definitions [2.4](#), [2.5](#), [2.6](#), [2.7](#), [2.10](#), [2.11](#), [2.12](#), [2.13](#), [2.14](#), [2.17](#), [2.18](#), [2.19](#), [2.20](#), [2.21](#), [2.24](#), [2.25](#), [2.26](#), [2.27](#), [2.28](#), [2.32](#), [2.33](#), [2.34](#), [2.37](#), [2.38](#), [2.39](#), [2.42](#), [2.43](#), [2.44](#), [2.45](#), [2.48](#), [2.49](#), [2.50](#), [2.51](#), [2.54](#), [2.55](#), [2.56](#), [2.57](#), [2.58](#), [2.61](#), [2.62](#), [2.64](#), [2.65](#), [2.66](#), [2.67](#), [2.68](#).

Search / Platform propositions/theorems: Propositions [2.8](#), [2.15](#), [2.22](#), [2.29](#), [2.30](#), [2.35](#), [2.40](#), [2.46](#), [2.52](#), [2.59](#), [2.63](#), [2.69](#), Theorem [2.70](#).

Operational cautions: Remarks [2.1](#), [2.2](#), [2.3](#), [2.9](#), [2.16](#), [2.23](#), [2.31](#), [2.36](#), [2.41](#), [2.47](#), [2.53](#), [2.60](#).

Explicit exclusions: Section V.14.

3 Appendix W: Bridge Programs

W.1. Scope and program status

This appendix defines Bridge Programs in the AK framework. It belongs to the Search / Platform Appendices. It does not enlarge the Core mathematical package of Chapters 1–8.

The central rule of this appendix is:

A Bridge Program is not a bridge theorem.

A Bridge Program organizes the tasks, hypotheses, artifacts, verification targets, failure modes, proof obligations, and promotion conditions required to turn a proposed cross-layer implication into a theorem. It does not itself prove that implication.

In particular, no Bridge Program, bridge draft, bridge candidate, bridge roadmap, dependency graph, agent recommendation, or proof-first planning artifact may replace:

$$B\text{-Gate}^+, \quad \text{Eval}_{1,W,\tau}(F) = 0, \quad C_{\tau,W}F, \quad \phi_{i,W,\tau}, \quad (\mu_{\text{Collapse}}, u_{\text{Collapse}}) = (0, 0), \quad \Sigma\delta < \text{gap}_{\tau}.$$

Remark 3.1 (Search / Platform status). Bridge Programs live in the Search / Platform layer. They may coordinate work across Core, Technical Extension, Problem Interface, and proof-first layers, but they do not change the claim status of any theorem by themselves.

Remark 3.2 (Relation to Appendices U and V). Appendix U defines agent semantics. Appendix V defines Hunter / Mapper / Lifter protocols. The present appendix defines how candidate bridges discovered, mapped, or lifted by those protocols are organized into auditable proof programs.

Remark 3.3 (Relation to Core re-entry). A Bridge Program may prepare material for Core re-entry or for a problem-interface soundness bridge. However, re-entry requires verifier-side data, including repaired evaluation records, admissible representatives when invoked, tower artifacts when invoked, diagnostics, gate verdicts, ledgers, and artifact references.

W.2. Bridge statements

Definition 3.4 (Bridge statement). A *bridge statement* is a statement connecting two semantic layers of the AK framework. Typical forms include:

external domain property $\implies$ Core-visible behavior,

Core certificate behavior $\implies$ external domain conclusion,

technical extension condition $\implies$ Core-readable certificate condition,

or

formal statement $\implies$ intended informal mathematical claim.

Definition 3.5 (Core-facing bridge statement). A *Core-facing bridge statement* is a bridge statement whose source or target involves verifier-side Core data, such as:

$$\text{Eval}_{i,W,\tau}(F), \quad C_{\tau,W}F, \quad \phi_{i,W,\tau}, \quad (\mu_{\text{Collapse}}, u_{\text{Collapse}}), \quad \Sigma\delta < \text{gap}_{\tau}.$$

Definition 3.6 (Problem-facing bridge statement). A *problem-facing bridge statement* is a bridge statement whose source or target is a problem-specific conclusion, such as arithmetic finiteness, Iwasawa control, PDE regularity, categorical equivalence, Fukaya-side unobstructedness, or cryptographic security.

Definition 3.7 (Bridge theorem). A *bridge theorem* is a bridge statement equipped with a complete proof under declared hypotheses, status labels, artifact references, and verifier-side reconstruction data.

Definition 3.8 (Bridge candidate). A *bridge candidate* is a bridge statement proposed for possible proof but not yet established as a bridge theorem.

Definition 3.9 (Bridge draft). A *bridge draft* is an informal, partial, agent-generated, exploratory, or schematic version of a bridge candidate.

Definition 3.10 (Bridge Program). A *Bridge Program* is a structured project record whose purpose is to turn one or more bridge candidates into bridge theorems or to classify why they fail.

Proposition 3.11 (Bridge candidate is not bridge theorem). *A bridge candidate is not a bridge theorem.*

Proof. A bridge candidate is a proposed implication. A bridge theorem requires a proof under declared hypotheses. Proposal and proof are distinct statuses by Appendix U. $\square$

Proposition 3.12 (Bridge draft is not bridge candidate unless specified). *A bridge draft is not a bridge candidate unless its statement, source semantics, target semantics, and intended status are explicitly specified.*

Proof. A draft may be informal, incomplete, ambiguous, or schematic. A bridge candidate requires a definite statement to be evaluated, mapped, decomposed, or proved. Thus a draft must be specified before it becomes a candidate. $\square$

Remark 3.13 (Bridge terminology caution). The word “bridge” should not be read as proof. Only a proved bridge theorem can support theorem-level transport across layers.

W.3. Bridge Program types

Definition 3.14 (Core-to-interface Bridge Program). A *Core-to-interface Bridge Program* attempts to prove a statement of the form:

$$\text{Core certificate behavior} \implies \text{problem-specific conclusion.}$$

Definition 3.15 (Interface-to-Core Bridge Program). An *Interface-to-Core Bridge Program* attempts to prove a statement of the form:

$$\text{problem-specific or external structure} \implies \text{Core-readable behavior.}$$

Definition 3.16 (Technical-to-Core Bridge Program). A *Technical-to-Core Bridge Program* attempts to prove that technical-extension data, such as discretization, measurement, controlled commutation, Mirror/Transfer comparison, derived realization, or higher-categorical structure, safely re-enter the Core verifier.

Definition 3.17 (Search-to-proof Bridge Program). A *Search-to-proof Bridge Program* attempts to convert search artifacts, candidate maps, lifted objects, bridge drafts, or advisory signals into proof objects, verified formal fragments, bridge theorems, or Core certificate candidates.

Definition 3.18 (Formal-to-informal Bridge Program). A *Formal-to-informal Bridge Program* attempts to prove that a verified formal fragment or proof assistant theorem corresponds to the intended informal mathematical statement.

Definition 3.19 (Problem-to-proof-first Bridge Program). A *Problem-to-proof-first Bridge Program* attempts to decompose a problem-specific theorem target into proof-first records, required bridge lemmas, Core certificate targets, analytic or arithmetic hypotheses, and proof object requirements.

Remark 3.20 (Bridge Program classes may overlap). A single Bridge Program may combine several types. For example, a Navier–Stokes regularity bridge may require interface-to-Core realization, technical-to-Core discretization soundness, Core-to-interface regularity transport, and formal-to-informal interpretation.

W.4. Bridge Program record

Definition 3.21 (Bridge Program record). A *Bridge Program record* contains:

- (i) Bridge Program identifier;
- (ii) bridge statement;

- (iii) Bridge Program type;
- (iv) source layer;
- (v) target layer;
- (vi) intended claim status;
- (vii) source semantics;
- (viii) target semantics;
- (ix) hypotheses;
- (x) required bridge lemmas;
- (xi) required artifacts;
- (xii) verification targets;
- (xiii) required Core-facing objects, if any;
- (xiv) required problem-facing objects, if any;
- (xv) defect and ledger treatment;
- (xvi) failure modes;
- (xvii) current status;
- (xviii) responsible agents or agent roles;
- (xix) dependency links to other programs;
- (xx) audit notes.

Definition 3.22 (Bridge Program status). A Bridge Program has one of the following statuses:

- (i) *proposed*;
- (ii) *specified*;
- (iii) *mapped*;
- (iv) *decomposed*;
- (v) *partially proved*;
- (vi) *blocked*;
- (vii) *failed*;
- (viii) *verified as bridge theorem*;
- (ix) *imported theorem*;
- (x) *rejected*;

- (xi) *superseded*;
- (xii) [Spec] *only*.

Definition 3.23 (Bridge Program claim status). The *claim status* of a Bridge Program is the strongest claim status justified by its current proof, artifact, and verification data. It may be lower than its intended status.

Proposition 3.24 (Program status is not theorem status). *A Bridge Program status does not imply theorem status unless the status is explicitly verified as bridge theorem or imported theorem and supported by proof data.*

Proof. Most Bridge Program statuses describe project progress, not mathematical proof. Only a verified theorem status or valid imported theorem status, backed by proof data and audit references, establishes theorem-level status. $\square$

Remark 3.25 (Status conservatism). Bridge Program status should be conservative. A partially proved, specified, mapped, or decomposed program must not be described as a theorem.

W.5. Bridge hypotheses

Definition 3.26 (Bridge hypothesis). A *bridge hypothesis* is an assumption required for a bridge statement to be meaningful or provable. Examples include:

- (i) realization faithfulness;
- (ii) constructibility;
- (iii) repaired evaluation compatibility;
- (iv) admissible representative existence;
- (v) eligible amplitude;
- (vi) edge realization;
- (vii) declared tower comparison artifact existence;
- (viii) diagnostic convergence;
- (ix) grid-to-continuum convergence;
- (x) measurement soundness;
- (xi) normalization transport;
- (xii) categorical comparison compatibility;
- (xiii) formal interpretation compatibility;
- (xiv) analytic regularity criteria;
- (xv) arithmetic control conditions.

Definition 3.27 (Bridge hypothesis status). Each bridge hypothesis must be labeled as:

- (i) proved;

- (ii) imported theorem;
- (iii) verified artifact;
- (iv) operational policy;
- (v) [Spec];
- (vi) unresolved;
- (vii) rejected;
- (viii) failed.

Definition 3.28 (Hypothesis discharge). A *hypothesis discharge* is a proof, valid import, verified artifact, or accepted operational policy that justifies a bridge hypothesis under the declared status.

Proposition 3.29 ([Spec] hypotheses prevent theorem-level bridge). A *bridge statement depending on an unresolved [Spec] hypothesis is not a bridge theorem*.

Proof. A theorem requires all necessary hypotheses and implications to be proved or validly imported. An unresolved [Spec] hypothesis is not proved. Therefore the bridge statement cannot be theorem-level. $\square$

Remark 3.30 (Spec is allowed but must be labeled). A Bridge Program may contain [Spec] hypotheses. This is useful for research planning. However, they block theorem-level status until discharged.

W.6. Bridge lemmas

Definition 3.31 (Bridge lemma). A *bridge lemma* is a smaller claim required to prove a bridge theorem.

Definition 3.32 (Core-facing bridge lemma). A *Core-facing bridge lemma* is a bridge lemma involving Core-readable data, such as repaired evaluation records, admissible representatives, eligible realizations, tower comparison artifacts, diagnostics, gates, or ledgers.

Definition 3.33 (Problem-facing bridge lemma). A *problem-facing bridge lemma* is a bridge lemma involving external semantic content, such as arithmetic, PDE, categorical, geometric, cryptographic, or formal-interpretation claims.

Definition 3.34 (Bridge lemma graph). A *bridge lemma graph* is a directed graph whose nodes are bridge lemmas and whose edges represent dependency relations among them.

Definition 3.35 (Discharge condition). A *discharge condition* is a criterion under which a bridge lemma is considered proved, imported, verified, rejected, or blocked.

Proposition 3.36 (All necessary bridge lemmas must be discharged). A *Bridge Program cannot yield a bridge theorem unless all necessary bridge lemmas are discharged*.

Proof. A bridge theorem depends on its required lemmas. If any necessary lemma is unresolved, the proof is incomplete. Therefore all necessary bridge lemmas must be discharged. $\square$

Remark 3.37 (Bridge lemma graph links to certificate DAG). The bridge lemma graph becomes part of the global certificate DAG described in the later validity-map and certificate-DAG appendix.

W.7. Core-facing bridge obligations

Definition 3.38 (Core-facing bridge obligation). A *Core-facing bridge obligation* is a required proof obligation connecting bridge data to verifier-side Core data. It may require:

- (i) construction of a Core-readable object;
- (ii) construction of repaired evaluation records

$$\text{Eval}_{i,w,\tau}(F);$$

- (iii) construction of admissible representative records

$$C_{\tau,w}F;$$

- (iv) proof of eligibility when the Ext^1 -clause is invoked;
- (v) construction of tower comparison artifacts

$$\phi_{i,w,\tau};$$

- (vi) computation or proof of monitored diagnostics;
- (vii) gate verdict support;
- (viii) ledger and budget support.

Definition 3.39 (Core bridge payload). A *Core bridge payload* is the collection of verifier-side Core data produced or required by a Bridge Program.

Proposition 3.40 (Core-facing bridge obligations do not verify themselves). A *Bridge Program listing Core-facing obligations does not verify those obligations*.

Proof. Listing an obligation records what must be proved or checked. Verification requires actual verifier-side data and the declared checking process. Therefore the obligation list is not self-verifying. $\square$

Remark 3.41 (Core bridge payload remains candidate until checked). A Core bridge payload produced by a Bridge Program remains candidate data until checked under the Core protocol.

W.8. Bridge defects and ledger integration

Definition 3.42 (Bridge defect). A *bridge defect* is a declared discrepancy, approximation, comparison cost, uncertainty, semantic mismatch, or transport loss introduced by a bridge construction.

Definition 3.43 (Bridge ledger component). A *bridge ledger component* is a ledger entry

$$\delta^{\text{bridge}}$$

recording the aggregate defect of a Bridge Program when its output is used in a certificate.

Definition 3.44 (Bridge-adjusted budget). If a certificate relies on a bridge with nonzero bridge defect, its adjusted budget is

$$\Sigma\delta_{\text{bridge}} := \Sigma\delta_{\text{base}} \oplus \delta^{\text{bridge}},$$

with aggregation interpreted in the declared ledger semantics.

Definition 3.45 (Bridge defect status). A bridge defect has one of the following statuses:

- (i) zero;
- (ii) bounded;
- (iii) ledgered;
- (iv) unbounded;
- (v) unresolved;
- (vi) failed.

Proposition 3.46 (Bridge defects affect Core only through ledger). *Bridge defects affect Core audit only if they are included in the declared δ -ledger.*

Proof. Bridge defects are not gate verdicts or monitored diagnostics. They affect Core audit only by contributing to the total budget. $\square$

Proposition 3.47 (Bridge-adjusted budget must pass). *A certificate relying on a nonzero bridge defect may pass Audit Mode only if*

$$\Sigma \delta_{\text{bridge}} < \text{gap}_{\tau}.$$

Proof. By Chapter 6, all deviations used by the certificate must be included in the ledger and the total must remain below the declared gap. $\square$

Remark 3.48 (Semantic mismatch is a defect). If a bridge changes semantics, weakens assumptions, strengthens conclusions, replaces a continuum object by a discrete object, or interprets a formal statement as an informal claim, the mismatch must be eliminated by theorem or ledgered as a defect.

W.9. Bridge Program and Core sovereignty

Theorem 3.49 (Bridge Programs do not update Core verdicts). *A Bridge Program does not update Core pass/reject unless it produces verifier-side data that are checked by the Core verifier.*

Proof. A Bridge Program is a search and proof-management object. By Core sovereignty, Core verdicts are determined only by verifier-side gates, diagnostics, repaired evaluation records, representative and eligibility records when invoked, tower artifacts when invoked, ledgers, and manifest data. Thus the program itself does not update Core verdicts. $\square$

Remark 3.50 (Bridge Program as coordination layer). A Bridge Program coordinates proof work. It does not bypass proof work.

W.10. Bridge Program failure modes

Definition 3.51 (Bridge Program failure). *A Bridge Program failure occurs when a Bridge Program cannot establish its intended bridge theorem under the declared hypotheses.*

Definition 3.52 (Hypothesis failure). *Hypothesis failure occurs when a required bridge hypothesis is false, unsupported, unavailable, or marked as failed.*

Definition 3.53 (Lemma failure). *Lemma failure occurs when a required bridge lemma cannot be proved, imported, verified, or discharged.*

Definition 3.54 (Realization failure). *Realization failure* occurs when the source object cannot be realized into the target layer in the required form.

Definition 3.55 (Core payload failure). *Core payload failure* occurs when the Bridge Program cannot produce required verifier-side Core data, such as:

$$\text{Eval}_{i,W,\tau}(F), \quad C_{\tau,W}F, \quad \phi_{i,W,\tau},$$

or the required diagnostic, gate, or ledger records.

Definition 3.56 (Transport failure). *Transport failure* occurs when the intended implication between source and target semantics cannot be established.

Definition 3.57 (Ledger failure). *Ledger failure* occurs when bridge defects cannot be bounded or when the bridge-adjusted budget fails:

$$\Sigma\delta_{\text{bridge}} \geq \text{gap}_{\tau}.$$

Definition 3.58 (Interpretation failure). *Interpretation failure* occurs when a formally verified or Core-verified statement cannot be validly interpreted as the intended external or problem-specific statement.

Definition 3.59 (Promotion failure). *Promotion failure* occurs when a Bridge Program cannot satisfy the evidence required to promote a bridge draft, bridge candidate, or [Spec] bridge to theorem-level status.

Proposition 3.60 (Bridge failure blocks theorem-level transport). *If a necessary Bridge Program fails, then the intended theorem-level transport is blocked.*

Proof. The intended transport depends on the bridge theorem. If the Bridge Program cannot establish that theorem, the transport implication is unavailable. $\square$

Remark 3.61 (Failure is informative). Bridge failure should be recorded precisely. It identifies whether the obstruction is in hypotheses, lemmas, realization, Core payload, transport, ledger, interpretation, or promotion.

W.11. Bridge Program lifecycle

Definition 3.62 (Bridge Program lifecycle). The *Bridge Program lifecycle* is:

propose $\longrightarrow$ specify $\longrightarrow$ decompose $\longrightarrow$ prove or import lemmas $\longrightarrow$ verify $\longrightarrow$ audit $\longrightarrow$ promote or reject.

Definition 3.63 (Promotion). *Promotion* is the act of changing a Bridge Program or bridge candidate to a higher status, such as from [Spec] to interface theorem or from interface theorem to imported Core theorem.

Definition 3.64 (Promotion condition). A *promotion condition* is the set of checks required before promotion. At minimum, it requires:

- (i) precise statement;
- (ii) complete hypotheses;
- (iii) proof or valid import;
- (iv) discharged necessary bridge lemmas;
- (v) artifact references;

- (vi) consistency with non-claims;
- (vii) ledger treatment of any defects;
- (viii) audit-readable record.

Definition 3.65 (Promotion record). A *promotion record* records the prior status, new status, evidence for promotion, approving verifier or auditor, artifact references, and any remaining limitations.

Proposition 3.66 (No silent promotion). *A bridge candidate or Bridge Program may not be silently promoted to theorem status.*

Proof. Promotion changes claim status. Claim status changes must be explicit, justified, and audit-readable. Silent promotion would violate the claim taxonomy and non-claim discipline. $\square$

Remark 3.67 (Promotion is rare). Most Bridge Programs remain research programs or [Spec] interfaces for long periods. This is acceptable and should be recorded honestly.

W.12. Bridge Programs and proof-first records

Definition 3.68 (Proof-first alignment). A Bridge Program is *proof-first aligned* if its bridge statement, hypotheses, lemmas, failure modes, Core payload, and promotion conditions are derived from a declared proof-first record.

Definition 3.69 (Proof-first dependency). A *proof-first dependency* is a dependency between a Bridge Program and a proof-first record, target theorem template, proof object schema, or theorem-level route.

Proposition 3.70 (Proof-first alignment prevents search inflation). *Proof-first alignment reduces the risk that search artifacts are mistaken for theorem-level bridges.*

Proof. A proof-first record specifies the target theorem and required bridges before search outputs are evaluated. Thus search artifacts are measured against predefined proof obligations rather than being inflated into claims after discovery. $\square$

Remark 3.71 (Relation to Appendix NS-C). Appendix NS-C is a major example of proof-first alignment. Its Navier–Stokes proof-first program decomposes the PDE theorem into bridge obligations.

W.13. Bridge Program manifest entries

Definition 3.72 (Bridge Program manifest entry). A *Bridge Program manifest entry* records:

- (i) Bridge Program identifier;
- (ii) bridge statement;
- (iii) Bridge Program type;
- (iv) source layer;
- (v) target layer;
- (vi) current status;
- (vii) intended claim status;

- (viii) hypotheses and their statuses;
- (ix) required lemmas and their statuses;
- (x) Core-facing obligations, if any;
- (xi) Core bridge payload, if any;
- (xii) problem-facing obligations, if any;
- (xiii) defect and ledger treatment;
- (xiv) dependency graph reference;
- (xv) proof-first dependency, if any;
- (xvi) responsible agents or roles;
- (xvii) artifact references;
- (xviii) failure modes, if any;
- (xix) promotion history.

Proposition 3.73 (Bridge manifest supports audit). *A Bridge Program manifest entry supports audit by recording the bridge statement, dependencies, hypotheses, proof status, Core-facing obligations, defects, and artifacts.*

Proof. Auditing requires reconstruction of what was claimed, what was assumed, what was proved, what remains blocked, and what artifacts support the record. The manifest entry supplies these data. $\square$

Remark 3.74 (Manifest does not prove bridge). A Bridge Program manifest records the state of the program. It does not prove the bridge unless it references a complete proof or verified proof object.

W.14. Bridge Program invariants

Definition 3.75 (Bridge Program invariant). *A Bridge Program invariant is a rule that must remain true throughout the Bridge Program lifecycle.*

Definition 3.76 (Non-theorem invariant). *The non-theorem invariant states that a Bridge Program is not theorem-level until a bridge theorem is proved or validly imported.*

Definition 3.77 (Hypothesis-label invariant). *The hypothesis-label invariant states that every required bridge hypothesis must have an explicit status label.*

Definition 3.78 (Lemma-discharge invariant). *The lemma-discharge invariant states that theorem-level bridge status requires discharge of all necessary bridge lemmas.*

Definition 3.79 (Core-payload invariant). *The Core-payload invariant states that any Core-facing bridge payload remains candidate data until checked by the declared Core verifier.*

Definition 3.80 (No-silent-promotion invariant). *The no-silent-promotion invariant states that every promotion in bridge status must be explicit, justified, and audit-readable.*

Proposition 3.81 (Bridge Program invariants prevent bridge inflation). *The Bridge Program invariants prevent bridge drafts, bridge candidates, and Bridge Programs from being silently inflated into theorem-level bridge claims.*

Proof. The non-theorem invariant blocks program-to-theorem conflation. The hypothesis-label invariant exposes unproved assumptions. The lemma-discharge invariant blocks incomplete proof graphs. The Core-payload invariant blocks unchecked Core-facing data. The no-silent-promotion invariant blocks unsupported claim upgrades. Together, they prevent bridge inflation. $\square$

W.15. Summary of Bridge Programs

Theorem 3.82 (Appendix W Bridge Program package). *Within the Search / Platform layer, the following package is available.*

- (i) *Bridge statements connect semantic layers, but are not theorems by default.*
- (ii) *Bridge drafts are not bridge candidates unless specified.*
- (iii) *Bridge candidates are not bridge theorems.*
- (iv) *A Bridge Program organizes proof obligations for bridge candidates.*
- (v) *Bridge Program status is not theorem status unless explicitly verified.*
- (vi) *[Spec] hypotheses prevent theorem-level bridge status until discharged.*
- (vii) *All necessary bridge lemmas must be discharged before bridge theorem status.*
- (viii) *Core-facing bridge obligations may involve*

$$\text{Eval}_{i,w,\tau}, \quad C_{\tau,w}F, \quad \phi_{i,w,\tau},$$

but listing such obligations does not verify them.

- (ix) *Core bridge payloads remain candidate data until checked.*
- (x) *Bridge defects affect Core audit only through the declared ledger.*
- (xi) *Bridge-adjusted budgets must satisfy*

$$\Sigma \delta_{\text{bridge}} < \text{gap}_{\tau}.$$
- (xii) *Bridge Programs do not update Core verdicts.*
- (xiii) *Bridge Program failures block theorem-level transport.*
- (xiv) *Bridge candidates may not be silently promoted.*
- (xv) *Proof-first alignment reduces search-driven claim inflation.*
- (xvi) *Bridge Program manifest entries support audit but do not themselves prove bridges.*
- (xvii) *Bridge Program invariants prevent bridge inflation.*

Proof. Items (i)–(iv) follow from the bridge statement, bridge draft, bridge candidate, bridge theorem, and Bridge Program definitions, together with Propositions 3.11 and 3.12. Item (v) is Proposition 3.24. Item (vi) is Proposition 3.29. Item (vii) is Proposition 3.36. Items (viii)–(ix) follow from the Core-facing bridge obligation definitions and Proposition 3.40. Items (x)–(xi) are Propositions 3.46 and 3.47. Item (xii) is Theorem 3.49. Item (xiii) is Proposition 3.60. Item (xiv) is Proposition 3.66. Item (xv) is Proposition 3.70. Item (xvi) follows from Proposition 3.73 and Remark 3.74. Item (xvii) is Proposition 3.81. $\square$

W.16. Explicit exclusions

The following are not asserted in Appendix W.

- No Bridge Program is treated as a bridge theorem.
- No bridge draft is treated as a specified bridge candidate.
- No bridge candidate is treated as proved.
- No [Spec] bridge is treated as theorem-level.
- No unresolved bridge hypothesis is treated as discharged.
- No partial bridge lemma graph is treated as complete proof.
- No Bridge Program status is treated as theorem status without proof.
- No Core-facing obligation is treated as verified by being listed.
- No Core bridge payload is treated as Core-verified without Core verification.
- No bridge defect is ignored when used in certification.
- No bridge-adjusted budget is bypassed.
- No Bridge Program updates Core pass/reject by itself.
- No failed Bridge Program supports theorem-level transport.
- No search-generated bridge is promoted silently.
- No proof-first record is replaced by a Bridge Program.
- No Bridge Program replaces B-Gate⁺.
- No Bridge Program replaces the repaired topological condition

$$\text{Eval}_{1,W,\tau}(F) = 0.$$

- No Bridge Program replaces admissible representative records

$$C_{\tau,W}F.$$

- No Bridge Program replaces monitored Type IV diagnostics.
- No Bridge Program replaces tower comparison artifacts

$$\phi_{i,W,\tau}.$$

- No Bridge Program replaces the budget condition

$$\Sigma\delta < \text{gap}_{\tau}.$$

W.17. Provenance and status

The material in this appendix depends on:

- (i) the Agent Semantics of Appendix U;
- (ii) the Hunter / Mapper / Lifter protocols of Appendix V;
- (iii) the repaired evaluation discipline of Appendix A;
- (iv) the admissible representative discipline of Appendix B;
- (v) the eligible realization regime of Appendix C;
- (vi) the monitored Type IV diagnostic and tower artifact discipline of Appendix D;
- (vii) the Core sovereignty doctrine of Chapter 6;
- (viii) the non-claim discipline of Chapter 7;
- (ix) the interface protocol of Chapter 8;
- (x) the manifest schema of Appendix G;
- (xi) the ledger catalog of Appendix M;
- (xii) the Problem Interface Appendices of Part IV;
- (xiii) the proof-first discipline of Appendix NS-C.

Its role is to define how proposed cross-layer implications are managed as auditable proof programs without being confused with proved theorems.

Status labels for Appendix W. *Search / Platform definitions:* Definitions [3.4](#), [3.5](#), [3.6](#), [3.7](#), [3.8](#), [3.9](#), [3.10](#), [3.14](#), [3.15](#), [3.16](#), [3.17](#), [3.18](#), [3.19](#), [3.21](#), [3.22](#), [3.23](#), [3.26](#), [3.27](#), [3.28](#), [3.31](#), [3.32](#), [3.33](#), [3.34](#), [3.35](#), [3.38](#), [3.39](#), [3.42](#), [3.43](#), [3.44](#), [3.45](#), [3.51](#), [3.52](#), [3.53](#), [3.54](#), [3.55](#), [3.56](#), [3.57](#), [3.58](#), [3.59](#), [3.62](#), [3.63](#), [3.64](#), [3.65](#), [3.68](#), [3.69](#), [3.72](#), [3.75](#), [3.76](#), [3.77](#), [3.78](#), [3.79](#), [3.80](#).

Search / Platform propositions/theorems: Propositions [3.11](#), [3.12](#), [3.24](#), [3.29](#), [3.36](#), [3.40](#), [3.46](#), [3.47](#), [3.60](#), [3.66](#), [3.70](#), [3.73](#), [3.81](#), Theorems [3.49](#), [3.82](#).

Operational cautions: Remarks [3.1](#), [3.2](#), [3.3](#), [3.13](#), [3.20](#), [3.25](#), [3.30](#), [3.37](#), [3.41](#), [3.48](#), [3.50](#), [3.61](#), [3.67](#), [3.71](#), [3.74](#).

Explicit exclusions: Section W.16.

4 Appendix X: Validity Map and Global Certificate DAG

X.1. Scope and map status

This appendix defines the Validity Map and the Global Certificate DAG used in the AK framework. It belongs to the Search / Platform Appendices. It does not enlarge the Core mathematical package of Chapters 1–8.

The central rule of this appendix is:

A Validity Map is navigation, not a theorem.

Likewise,

A Certificate DAG organizes proof dependencies; it does not prove them by existing.

In particular, no node, edge, path, map region, graph layout, dependency chain, DAG visualization, search-map structure, or certificate skeleton may replace:

$$B\text{-Gate}^+, \quad \text{Eval}_{1,W,\tau}(F) = 0, \quad C_{\tau,W}F, \quad \phi_{i,W,\tau}, \quad (\mu_{\text{Collapse}}, u_{\text{Collapse}}), \quad \Sigma\delta < \text{gap}_{\tau}.$$

Remark 4.1 (Search / Platform status). Validity Maps and Certificate DAGs are platform-level organization devices. They support navigation, audit, dependency tracking, proof planning, status control, and reproducibility. They are not theorem-generating mechanisms.

Remark 4.2 (Relation to Appendices U, V, and W). Appendix U defines agent semantics. Appendix V defines Hunter / Mapper / Lifter search protocols. Appendix W defines Bridge Programs and bridge lemma graphs. The present appendix specifies how those objects may be assembled into global validity maps and target-specific certificate dependency structures.

Remark 4.3 (Map and DAG are governance structures). A Validity Map governs status and navigation. A Certificate DAG governs dependency and discharge. Neither governs truth unless the referenced proof, verification, artifact, and audit data are complete.

X.2. Validity Map

Definition 4.4 (Validity Map). A *Validity Map* is a structured map of claims, candidates, bridges, certificate fragments, proof objects, artifacts, failure modes, and proof statuses in an AK workflow. It records where claims stand relative to verification, audit, bridge discharge, and theorem-level readiness.

Definition 4.5 (Validity Map node). A *Validity Map node* is an entry representing one of:

- (i) a Core claim;
- (ii) a Core-facing object;
- (iii) a repaired evaluation record;
- (iv) an admissible representative record;
- (v) an eligible realization record;
- (vi) a tower comparison artifact;
- (vii) a diagnostic result;
- (viii) a ledger component;
- (ix) an interface claim;
- (x) a bridge draft;
- (xi) a bridge candidate;

- (xii) a Bridge Program;
- (xiii) a bridge theorem;
- (xiv) a search artifact;
- (xv) a formal fragment;
- (xvi) a verified formal fragment;
- (xvii) a certificate candidate;
- (xviii) a verified certificate;
- (xix) a proof-first record;
- (xx) a proof object;
- (xxi) a failure mode;
- (xxii) a non-claim;
- (xxiii) a [Spec] statement.

Definition 4.6 (Core-facing map node). A *Core-facing map node* is a Validity Map node involving verifier-side Core data, including:

$$\text{Eval}_{i,W,\tau}(F), \quad C_{\tau,W}F, \quad \phi_{i,W,\tau}, \quad (\mu_{\text{Collapse}}, u_{\text{Collapse}}), \quad \Sigma\delta < \text{gap}_{\tau},$$

or the gate verdicts and manifest entries needed to support them.

Definition 4.7 (Validity Map edge). A *Validity Map edge* is a typed relation between two Validity Map nodes. Permitted edge types include:

- (i) depends-on;
- (ii) supports;
- (iii) refines;
- (iv) blocks;
- (v) contradicts;
- (vi) specializes;
- (vii) generalizes;
- (viii) imports;
- (ix) discharges;
- (x) suggests;
- (xi) cites-artifact;
- (xii) verifies;

- (xiii) audits;
- (xiv) realizes;
- (xv) lifts-to;
- (xvi) normalizes-to;
- (xvii) interprets-as;
- (xviii) ledger-contributes-to;
- (xix) bridge-transport-to;
- (xx) formalizes;
- (xxi) deprecates;
- (xxii) supersedes.

Proposition 4.8 (Validity Map edges must be typed). *Every edge in a Validity Map must have a declared edge type.*

Proof. Untyped edges are ambiguous. They may be misread as implication, dependency, evidence, analogy, artifact reference, theorem, or verified transport. Typed edges prevent claim inflation and support audit. $\square$

Remark 4.9 (Suggests is not implies). An edge labeled *suggests* is advisory. It must not be read as logical implication.

X.3. Validity statuses

Definition 4.10 (Validity status). A *validity status* is a label attached to a node describing its current verification state. Permitted statuses include:

- (i) search artifact;
- (ii) candidate;
- (iii) draft;
- (iv) advisory;
- (v) [Spec];
- (vi) operational policy;
- (vii) interface specification;
- (viii) bridge draft;
- (ix) bridge candidate;
- (x) Bridge Program;
- (xi) partially proved;
- (xii) blocked;

- (xiii) failed;
- (xiv) rejected;
- (xv) formal fragment;
- (xvi) verified formal fragment;
- (xvii) Core certificate candidate;
- (xviii) Core-facing candidate;
- (xix) Core verified;
- (xx) interface theorem;
- (xxi) bridge theorem;
- (xxii) imported theorem;
- (xxiii) proof-first record;
- (xxiv) proof object candidate;
- (xxv) theorem-level proof object;
- (xxvi) non-claim;
- (xxvii) deprecated;
- (xxviii) superseded.

Definition 4.11 (Status transition). A *status transition* is a recorded change from one validity status to another.

Definition 4.12 (Status transition rule). A *status transition rule* specifies what evidence, verification, audit, proof, import, discharge, or demotion condition is required before a status transition is allowed.

Definition 4.13 (Promotion transition). A *promotion transition* is a status transition to a stronger claim status. Examples include:

bridge candidate $\longrightarrow$ bridge theorem,
Core certificate candidate $\longrightarrow$ Core verified,
proof object candidate $\longrightarrow$ theorem-level proof object.

Definition 4.14 (Demotion transition). A *demotion transition* is a status transition to a weaker or safer claim status due to failure, artifact drift, missing dependencies, invalid proof, or supersession.

Proposition 4.15 (Status transitions require evidence). A node may not be promoted to a stronger validity status unless the required transition evidence is recorded.

Proof. Validity status affects how a node may be used in proof planning, certification, bridge transport, and theorem-level claims. Promotion without evidence would allow unsupported claim inflation. Therefore transition evidence is required. $\square$

Remark 4.16 (Demotion is allowed). Nodes may be demoted when failures, gaps, artifact drift, interpretation errors, or dependency invalidation are discovered. Demotion protects the integrity of the map.

X.4. Global Certificate DAG

Definition 4.17 (Certificate DAG). A *Certificate DAG* is a directed acyclic graph whose nodes are certificate-relevant objects and whose edges record typed dependency relations among them.

Definition 4.18 (Global Certificate DAG). The *Global Certificate DAG* is the certificate-level DAG that organizes all Core, interface, bridge, technical-extension, proof-first, formalization, ledger, artifact, and platform dependencies relevant to a target proof object or target certificate.

Definition 4.19 (DAG node). A *DAG node* may represent:

- (i) a definition;
- (ii) a lemma;
- (iii) a theorem;
- (iv) a bridge statement;
- (v) a Bridge Program;
- (vi) a bridge theorem;
- (vii) a repaired evaluation record;
- (viii) an admissible representative record;
- (ix) an eligibility record;
- (x) a tower comparison artifact;
- (xi) a diagnostic result;
- (xii) a certificate fragment;
- (xiii) a ledger component;
- (xiv) a gate verdict;
- (xv) a formal fragment;
- (xvi) a proof object;
- (xvii) an artifact reference;
- (xviii) a failure condition;
- (xix) an imported theorem;
- (xx) an interpretation theorem.

Definition 4.20 (Core DAG node). A *Core DAG node* is a DAG node representing verifier-side Core content, including:

$$\text{Eval}_{i,W,\tau}(F), \quad C_{\tau,W}F, \quad \phi_{i,W,\tau}, \quad B\text{-Gate}^+, \quad (\mu_{\text{Collapse}}, u_{\text{Collapse}}), \quad \Sigma\delta < \text{gap}_{\tau}.$$

Definition 4.21 (DAG edge). A *DAG edge* is a typed dependency from one node to another. It must declare whether it represents logical dependency, artifact dependency, verification dependency, ledger dependency, diagnostic dependency, bridge dependency, interpretation dependency, formalization dependency, or audit dependency.

Proposition 4.22 (DAG existence is not proof). *The existence of a Certificate DAG does not prove the target claim.*

Proof. A DAG records dependencies and statuses. The target claim is proved only if the required nodes are proved or verified and all necessary dependency edges are discharged according to their semantics. The graph structure alone does not establish those conditions. $\square$

Remark 4.23 (DAG is proof skeleton). A Certificate DAG is best understood as a proof skeleton or audit skeleton. It becomes proof-supporting only when its necessary nodes and edges are sufficiently discharged.

X.5. Acyclicity

Definition 4.24 (DAG acyclicity condition). A Certificate DAG satisfies the *acyclicity condition* if it has no directed cycles among ordinary dependency edges.

Definition 4.25 (Dependency cycle). A *dependency cycle* is a directed cycle in which a node depends, directly or indirectly, on itself.

Definition 4.26 (Justified cyclic proof principle). A *justified cyclic proof principle* is a separately stated fixed-point, mutual-induction, coinduction, guarded recursion, or circular-proof principle that permits a controlled cyclic dependency.

Proposition 4.27 (Dependency cycles block DAG certification). *A dependency cycle blocks ordinary DAG certification unless the cycle is replaced by a separately justified fixed-point, mutual-induction, coinductive, guarded, or otherwise accepted proof principle.*

Proof. Ordinary DAG-based certification requires acyclic dependency order. A cycle prevents topological ordering of proof obligations. It can be allowed only if replaced by a justified proof principle that explicitly handles cyclic dependence. $\square$

Remark 4.28 (No hidden circular proof). The DAG discipline is designed to expose circular reasoning. If a circular structure is mathematically valid, its justification must be explicit.

X.6. Edge semantics

Definition 4.29 (Logical edge). A *logical edge*

$$A \rightarrow B$$

means that B logically depends on A .

Definition 4.30 (Artifact edge). An *artifact edge*

$$A \rightarrow B$$

means that B references or requires artifact A .

Definition 4.31 (Ledger edge). A *ledger edge*

$$A \rightarrow B$$

means that ledger component A contributes to the budget or audit condition of B .

Definition 4.32 (Diagnostic edge). A *diagnostic edge*

$$A \rightarrow B$$

means that diagnostic result A is required for certificate node B .

Definition 4.33 (Bridge edge). A *bridge edge*

$$A \rightarrow B$$

means that bridge theorem or bridge candidate A is used to connect the semantics of B to another layer. Its status must specify whether A is theorem-level, [Spec], candidate, failed, or imported.

Definition 4.34 (Interpretation edge). An *interpretation edge*

$$A \rightarrow B$$

means that formal, Core, or platform-side statement A is interpreted as supporting informal or problem-specific claim B . Such an edge requires an interpretation theorem or explicit [Spec] status.

Definition 4.35 (Formalization edge). A *formalization edge*

$$A \rightarrow B$$

means that informal statement A is represented by formal fragment or formal theorem B .

Definition 4.36 (Advisory edge). An *advisory edge*

$$A \rightarrow B$$

means that A suggests, motivates, or prioritizes B , but does not logically support B .

Proposition 4.37 (Advisory edges are non-proof edges). *Advisory edges do not discharge proof obligations.*

Proof. An advisory edge records motivation or navigation. It is not a logical, diagnostic, ledger, verified bridge, interpretation, or formal proof dependency. Therefore it cannot discharge a proof obligation. $\square$

Remark 4.38 (Edge typing prevents misuse). A single visual arrow can be misleading. The typed edge system prevents advisory, artifact, formalization, interpretation, bridge, and logical relations from being conflated.

X.7. Node discharge

Definition 4.39 (Discharged node). A node is *discharged* if its required verification, proof, artifact, diagnostic, ledger, bridge, interpretation, formalization, or audit condition has been satisfied.

Definition 4.40 (Undischarged node). A node is *undischarged* if at least one necessary condition remains unresolved.

Definition 4.41 (Discharge certificate). A *discharge certificate* is a record explaining why a node is considered discharged. It must include:

- (i) node identifier;
- (ii) node type;
- (iii) required conditions;
- (iv) evidence supplied;
- (v) verifier or verifying agent, if any;
- (vi) verification environment, if any;
- (vii) artifact references;
- (viii) status label after discharge;
- (ix) limitations or residual [Spec] assumptions, if any.

Definition 4.42 (Core node discharge). A *Core node discharge* is the discharge of a Core DAG node through the relevant verifier-side data, such as repaired evaluation records, admissible representatives, eligibility records, tower comparison artifacts, monitored diagnostics, gate verdicts, and ledger checks.

Proposition 4.43 (Target node requires all necessary dependencies discharged). *A target node in the Global Certificate DAG may be considered discharged only if all necessary dependency nodes and dependency edges are discharged.*

Proof. The target depends on its necessary dependency structure. If any necessary dependency remains unresolved, the target proof is incomplete. Therefore all necessary dependencies must be discharged. $\square$

Proposition 4.44 (Core node discharge requires Core verifier data). *A Core DAG node cannot be discharged merely by map status, agent assertion, or DAG placement. It requires the relevant Core verifier data.*

Proof. Core DAG nodes represent Core-facing objects or verdicts. By Core sovereignty and Appendix U, such nodes require verifier-side support. Map placement, agent assertion, or dependency layout does not supply that support. $\square$

Remark 4.45 (Optional support is different). Optional advisory or explanatory dependencies need not be discharged for theorem-level status unless explicitly marked as necessary.

X.8. Global certificate assembly

Definition 4.46 (Global certificate assembly). *Global certificate assembly* is the process of collecting discharged nodes, verified edges, ledger records, diagnostics, bridge theorems, interpretation theorems, and artifacts into a target certificate record or theorem-level proof object.

Definition 4.47 (Assembly policy). An *assembly policy* declares:

- (i) target node;
- (ii) required dependency subgraph;
- (iii) necessary and optional dependencies;
- (iv) discharge conditions;
- (v) ledger aggregation policy;
- (vi) diagnostic inclusion policy;
- (vii) bridge inclusion policy;
- (viii) interpretation inclusion policy;
- (ix) artifact completeness policy;
- (x) audit requirements;
- (xi) failure and demotion rules.

Definition 4.48 (Certificate assembly payload). A *certificate assembly payload* is the collection of records assembled for the target certificate, including repaired evaluation records, representative records, eligibility records, tower artifacts, diagnostics, ledgers, bridge theorem references, artifact references, and manifest entries.

Proposition 4.49 (Global assembly requires manifest completeness). *Global certificate assembly is admissible only if the resulting certificate record satisfies the Minimal Manifest Axiom.*

Proof. A global certificate is still a certificate record. By Appendix G, any certificate record must include the required declarations, repaired evaluation records, threshold and collapse policies, ledger policy, diagnostics, gate verdicts, and artifact references. Therefore manifest completeness is required. $\square$

Remark 4.50 (Assembly is not invention). Assembly collects and organizes already verified, proved, declared, or explicitly status-labeled components. It does not invent missing proof obligations.

X.9. Validity Map versus Certificate DAG

Definition 4.51 (Map-DAG distinction). The *Validity Map* is a broad navigation and status structure. The *Global Certificate DAG* is a proof-dependency structure for a specific target certificate or proof object.

Proposition 4.52 (Validity Map is broader than Certificate DAG). *A Validity Map may contain nodes and edges that are not part of the Global Certificate DAG for a given target.*

Proof. A Validity Map includes candidates, rejected routes, advisory relations, search artifacts, failed bridge programs, deprecated nodes, and [Spec] statements. A Certificate DAG includes the dependencies relevant to a specific certificate target. Thus the map is broader. $\square$

Definition 4.53 (Target sub-DAG). A *target sub-DAG* is the dependency subgraph of the Global Certificate DAG required to support a specific target node.

Remark 4.54 (Do not over-certify the map). Only the target certificate sub-DAG requires discharge for a given certificate. The entire Validity Map need not be fully discharged.

X.10. Failure and demotion

Definition 4.55 (Map failure). A *Map failure* occurs when a Validity Map node, edge, status label, artifact reference, or transition record is incorrect, ambiguous, stale, unsupported, or inconsistent.

Definition 4.56 (DAG failure). A *DAG failure* occurs when a Certificate DAG contains an unresolved necessary dependency, invalid edge, undischarged required node, forbidden cycle, inconsistent status, missing artifact, failed bridge, invalid interpretation edge, or unsupported Core node.

Definition 4.57 (Demotion event). A *demotion event* is a recorded downgrade of a node, edge, subgraph, map region, or certificate status due to discovered failure, artifact drift, unresolved dependency, invalid interpretation, or bridge failure.

Definition 4.58 (Subgraph quarantine). *Subgraph quarantine* is the temporary isolation of a map or DAG subgraph whose status is uncertain, inconsistent, stale, or under review.

Proposition 4.59 (DAG failure blocks target certification). *A DAG failure in a necessary dependency subgraph blocks target certification until repaired.*

Proof. A necessary dependency subgraph is required for the target certificate. If it contains failure, the target certificate is unsupported. Repair, replacement, demotion, or removal of the failed dependency is required. $\square$

Remark 4.60 (Demotion protects integrity). Demotion is not a failure of the platform. It is a safety mechanism that prevents stale, invalid, or over-promoted claims from remaining active at too strong a status.

X.11. Artifact consistency

Definition 4.61 (Artifact consistency). A node or edge has *artifact consistency* if every referenced artifact exists, has the declared identity, has the declared version, and supports the claim or relation for which it is cited.

Definition 4.62 (Artifact drift). *Artifact drift* occurs when a referenced artifact changes, disappears, is superseded, becomes inconsistent, or no longer supports the node or edge that cites it.

Definition 4.63 (Artifact identity record). An *artifact identity record* is a record of an artifact's identifier, version, hash or equivalent integrity marker, source, status, and supported nodes or edges.

Definition 4.64 (Artifact support relation). An *artifact support relation* declares which claim, node, edge, diagnostic, ledger, proof, or certificate component an artifact supports.

Proposition 4.65 (Artifact drift requires review). *If artifact drift is detected in a necessary dependency, the affected node or edge must be reviewed and may need demotion.*

Proof. Artifact drift may invalidate the evidence supporting a node or edge. Until reviewed, the dependency is not audit-stable. Thus review and possible demotion are required. $\square$

Remark 4.66 (Hashing and versioning). Hashing, versioning, immutable identifiers, and proof-store references are platform mechanisms for controlling artifact drift. They are developed further in the execution and reproducibility appendices.

X.12. Bridge Programs inside the DAG

Definition 4.67 (Bridge Program DAG node). A *Bridge Program DAG node* is a DAG node representing a Bridge Program, bridge candidate, bridge theorem, bridge lemma graph, or bridge discharge record.

Definition 4.68 (Bridge theorem discharge edge). A *bridge theorem discharge edge* is an edge showing that a bridge theorem discharges a bridge dependency required by a target node.

Definition 4.69 (Bridge Program blocked edge). A *Bridge Program blocked edge* is an edge showing that an unresolved, failed, or [Spec] Bridge Program blocks theorem-level transport.

Proposition 4.70 (Bridge Program node does not discharge bridge dependency by existence). A *Bridge Program DAG node does not discharge a bridge dependency unless it is supported by a bridge theorem, valid import, or verified discharge record.*

Proof. By Appendix W, a Bridge Program is not a bridge theorem. Therefore its presence in the DAG records a proof program or dependency, but does not discharge the dependency by itself. $\square$

Remark 4.71 (Bridge Program placement). Bridge Programs are useful DAG nodes because they expose what must be proved. Their status must remain distinct from bridge theorem nodes.

X.13. Formalization and interpretation inside the DAG

Definition 4.72 (Formalization DAG node). A *formalization DAG node* is a node representing a formal statement, formal fragment, proof assistant file, verified formal theorem, or formal environment.

Definition 4.73 (Interpretation theorem node). An *interpretation theorem node* is a node representing a theorem that connects a verified formal statement to the intended informal or problem-specific claim.

Proposition 4.74 (Verified formal node does not imply intended claim without interpretation). A *verified formal node does not discharge an intended informal or problem-specific claim unless the required interpretation edge is discharged.*

Proof. A proof assistant verifies the formal statement in its formal environment. The intended informal claim may have different semantics, hypotheses, or scope. An interpretation theorem is required to connect them. $\square$

Remark 4.75 (Formalization mismatch tracking). Formalization mismatch should appear in the Validity Map and Certificate DAG as either an undischarged interpretation edge, a blocked bridge, or a demotion event.

X.14. Validity Map manifest entries

Definition 4.76 (Validity Map manifest entry). A *Validity Map manifest entry* records:

- (i) map identifier;
- (ii) node set;
- (iii) edge set;
- (iv) node status labels;
- (v) edge types;

- (vi) Core-facing nodes;
- (vii) Bridge Program nodes;
- (viii) formalization and interpretation nodes;
- (ix) artifact references;
- (x) status transition history;
- (xi) demotion events;
- (xii) quarantined subgraphs, if any;
- (xiii) audit notes.

Definition 4.77 (Certificate DAG manifest entry). A *Certificate DAG manifest entry* records:

- (i) DAG identifier;
- (ii) target node;
- (iii) required dependency subgraph;
- (iv) node discharge statuses;
- (v) edge discharge statuses;
- (vi) Core node discharge records;
- (vii) ledger dependencies;
- (viii) diagnostic dependencies;
- (ix) bridge dependencies;
- (x) interpretation dependencies;
- (xi) artifact references;
- (xii) assembly policy;
- (xiii) audit result.

Proposition 4.78 (Map and DAG manifests support audit). *Validity Map and Certificate DAG manifest entries support audit by recording structure, status, dependencies, artifacts, discharge conditions, and demotion history.*

Proof. Audit requires reconstructing what was used, how it was connected, which dependencies were necessary, and whether those dependencies were discharged. The map and DAG manifests record these data. $\square$

Remark 4.79 (Manifest is not proof). A map or DAG manifest records structure and status. It does not prove the target unless the referenced proof, verification, interpretation, and artifact data are complete.

X.15. Map and DAG invariants

Definition 4.80 (Map-DAG invariant). A *Map-DAG invariant* is a rule that must remain true across Validity Map and Certificate DAG updates.

Definition 4.81 (Typed-edge invariant). The *typed-edge invariant* states that every map or DAG edge must have declared semantics.

Definition 4.82 (Evidence-promotion invariant). The *evidence-promotion invariant* states that no node may be promoted to a stronger status without recorded evidence.

Definition 4.83 (Core-sovereignty invariant). The *Core-sovereignty invariant* states that no map or DAG structure may replace Core verifier conditions.

Definition 4.84 (Bridge-status invariant). The *bridge-status invariant* states that bridge drafts, bridge candidates, Bridge Programs, and bridge theorem nodes must remain status-distinct.

Definition 4.85 (Artifact-consistency invariant). The *artifact-consistency invariant* states that necessary nodes and edges must maintain artifact consistency or be reviewed and possibly demoted.

Proposition 4.86 (Map-DAG invariants prevent proof inflation). *The typed-edge, evidence-promotion, Core-sovereignty, bridge-status, and artifact-consistency invariants prevent navigation structures from being inflated into proof.*

Proof. Typed edges prevent ambiguous arrows. Evidence-promotion blocks unsupported status upgrades. Core sovereignty blocks map/DAG replacement of verifier-side conditions. Bridge-status separation blocks Bridge Program inflation. Artifact consistency blocks stale dependency support. Together, these invariants preserve proof integrity. $\square$

X.16. Summary of Validity Map and Global Certificate DAG

Theorem 4.87 (Appendix X map-DAG package). *Within the Search / Platform layer, the following package is available.*

- (i) *A Validity Map is navigation, not theorem.*
- (ii) *A Certificate DAG organizes proof dependencies, but does not prove them merely by existing.*
- (iii) *Validity Map nodes and edges must be typed and status-labeled.*
- (iv) *Core-facing nodes may represent*

$$\text{Eval}_{i,W,\tau}, \quad C_{\tau,W}F, \quad \phi_{i,W,\tau}, \quad (\mu_{\text{Collapse}}, u_{\text{Collapse}}), \quad \Sigma\delta < \text{gap}_{\tau},$$
but map placement does not verify them.
- (v) *Status promotion requires recorded evidence.*
- (vi) *Dependency cycles block ordinary DAG certification unless explicitly justified.*
- (vii) *Advisory edges do not discharge proof obligations.*
- (viii) *Core node discharge requires Core verifier data.*
- (ix) *A target node requires all necessary dependency nodes and edges to be discharged.*

- (x) *Global certificate assembly requires Minimal Manifest completeness.*
- (xi) *The Validity Map is broader than a target Certificate DAG.*
- (xii) *DAG failure in a necessary subgraph blocks target certification.*
- (xiii) *Artifact drift requires review and possible demotion.*
- (xiv) *Bridge Program nodes do not discharge bridge dependencies by existence.*
- (xv) *Verified formal nodes do not imply intended informal claims without discharged interpretation edges.*
- (xvi) *Map and DAG manifests support audit but do not themselves prove the target.*
- (xvii) *Map-DAG invariants prevent proof inflation.*

Proof. Items (i)–(ii) are Proposition 4.22 and the definition of the Validity Map. Item (iii) follows from the node, edge, and status definitions and Proposition 4.8. Item (iv) follows from Definitions 4.6 and 4.20, and Proposition 4.44. Item (v) is Proposition 4.15. Item (vi) is Proposition 4.27. Item (vii) is Proposition 4.37. Item (viii) is Proposition 4.44. Item (ix) is Proposition 4.43. Item (x) is Proposition 4.49. Item (xi) is Proposition 4.52. Item (xii) is Proposition 4.59. Item (xiii) is Proposition 4.65. Item (xiv) is Proposition 4.70. Item (xv) is Proposition 4.74. Item (xvi) follows from Proposition 4.78 and Remark 4.79. Item (xvii) is Proposition 4.86. □

X.17. Explicit exclusions

The following are not asserted in Appendix X.

- No Validity Map is treated as a theorem.
- No Certificate DAG is treated as proof merely by existing.
- No untyped edge is treated as logical implication.
- No advisory edge discharges a proof obligation.
- No node is promoted without recorded evidence.
- No dependency cycle is accepted without explicit justification.
- No undischarged target dependency is ignored.
- No artifact reference is assumed stable without version or consistency control.
- No map visualization is treated as proof.
- No DAG layout is treated as verification.
- No certificate assembly bypasses the Minimal Manifest Axiom.
- No failed necessary subgraph supports target certification.
- No Bridge Program node is treated as a bridge theorem.
- No bridge candidate node is treated as a discharged bridge dependency.

- No verified formal node is interpreted as an informal theorem without an interpretation theorem.
- No Core-facing node is treated as Core-verified merely by appearing in the map or DAG.
- No map or DAG node replaces B-Gate⁺.
- No map or DAG node replaces the repaired topological condition

$$\text{Eval}_{1,W,\tau}(F) = 0.$$

- No map or DAG node replaces admissible representative records

$$C_{\tau,W}F.$$

- No map or DAG node replaces monitored Type IV diagnostics.
- No map or DAG node replaces tower comparison artifacts

$$\phi_{i,W,\tau}.$$

- No Validity Map or Certificate DAG replaces the budget condition

$$\Sigma\delta < \text{gap}_{\tau}.$$

X.18. Provenance and status

The material in this appendix depends on:

- (i) the Agent Semantics of Appendix U;
- (ii) the Hunter / Mapper / Lifter protocols of Appendix V;
- (iii) the Bridge Programs of Appendix W;
- (iv) the repaired evaluation discipline of Appendix A;
- (v) the admissible representative discipline of Appendix B;
- (vi) the eligible realization regime of Appendix C;
- (vii) the monitored Type IV diagnostic and tower artifact discipline of Appendix D;
- (viii) the Core sovereignty doctrine of Chapter 6;
- (ix) the non-claim discipline of Chapter 7;
- (x) the manifest schema of Appendix G;
- (xi) the ledger catalog of Appendix M;
- (xii) the proof-first discipline of Appendix NS-C.

Its role is to provide the global map and DAG structure required for proof planning, dependency tracking, bridge governance, certificate assembly, and audit without confusing navigation with proof.

Status labels for Appendix X. *Search / Platform definitions:* Definitions 4.4, 4.5, 4.6, 4.7, 4.10, 4.11, 4.12, 4.13, 4.14, 4.17, 4.18, 4.19, 4.20, 4.21, 4.24, 4.25, 4.26, 4.29, 4.30, 4.31, 4.32, 4.33, 4.34, 4.35, 4.36, 4.39, 4.40, 4.41, 4.42, 4.46, 4.47, 4.48, 4.51, 4.53, 4.55, 4.56, 4.57, 4.58, 4.61, 4.62, 4.63, 4.64, 4.67, 4.68, 4.69, 4.72, 4.73, 4.76, 4.77, 4.80, 4.81, 4.82, 4.83, 4.84, 4.85.

Search / Platform propositions/theorems: Propositions 4.8, 4.15, 4.22, 4.27, 4.37, 4.43, 4.44, 4.49, 4.52, 4.59, 4.65, 4.70, 4.74, 4.78, 4.86, Theorem 4.87.

Operational cautions: Remarks 4.1, 4.2, 4.3, 4.9, 4.16, 4.23, 4.28, 4.38, 4.45, 4.50, 4.54, 4.60, 4.66, 4.71, 4.75, 4.79.

Explicit exclusions: Section X.17.

5 Appendix Y: AI Platform and Proof Store

Y.1. Scope and platform status

This appendix defines the AI Platform and Proof Store layer of the AK framework. It belongs to the Search / Platform Appendices. It does not enlarge the Core mathematical package of Chapters 1–8.

The central rule of this appendix is:

Storage is not certification.

A Proof Store may store, index, version, retrieve, compare, and audit proof-related objects. However, the fact that an object is stored does not imply that it is true, verified, Core-admissible, or theorem-level.

In particular, no platform entry, stored proof object, AI-generated artifact, claim registry item, version tag, proof-store identifier, hash, query result, or storage event may replace:

$$B\text{-Gate}^+, \quad \text{Eval}_{1,W,\tau}(F) = 0, \quad C_{\tau,W}F, \quad \phi_{i,W,\tau}, \quad (\mu_{\text{Collapse}}, u_{\text{Collapse}}), \quad \Sigma\delta < \text{gap}_{\tau}.$$

Remark 5.1 (Search / Platform status). This appendix concerns infrastructure for managing proof objects, artifacts, claims, agent outputs, verification records, audit records, and replay references. It does not create mathematical validity.

Remark 5.2 (Relation to Appendices U–X). Appendix U defines agent semantics. Appendix V defines search protocols. Appendix W defines Bridge Programs. Appendix X defines Validity Maps and Certificate DAGs. The present appendix defines the storage, registry, lifecycle, and platform layer supporting those structures.

Remark 5.3 (Storage supports governance). The platform layer is not a proof engine by default. It preserves identities, histories, statuses, and dependencies so that proof engines, auditors, and human reviewers can reconstruct what has been claimed and checked.

Y.2. AI Platform

Definition 5.4 (AI Platform). An *AI Platform* is a software, data, and workflow environment supporting agent-assisted search, proof management, artifact storage, verification, audit, replay, and reproducibility for the AK framework.

Definition 5.5 (Platform component). A *platform component* is a subsystem of the AI Platform. Examples include:

- (i) agent interface;
- (ii) search engine;
- (iii) proof assistant interface;
- (iv) persistence-computation engine;
- (v) artifact store;
- (vi) proof store;
- (vii) claim registry;
- (viii) validity map service;
- (ix) certificate DAG service;
- (x) ledger engine;
- (xi) diagnostic engine;
- (xii) replay engine;
- (xiii) audit dashboard;
- (xiv) access-control service.

Definition 5.6 (Platform action). A *platform action* is an operation performed by the AI Platform or one of its components. Examples include:

create, store, retrieve, version, hash, verify, audit, replay, promote, demote, reject, supersede.

Definition 5.7 (Platform authority scope). A *platform authority scope* is the declared scope within which a platform component or action is authorized to operate.

Proposition 5.8 (Platform action is scoped). A *platform action establishes only the effect declared by its action type and authority scope*.

Proof. Different platform actions have different meanings. Storing an object records it. Hashing identifies content. Verification checks declared criteria. Audit checks reconstructibility. Promotion changes status only under an authority rule. Therefore each action is scoped to its declared semantics. $\square$

Remark 5.9 (Platform semantics must be explicit). The same interface button or workflow step may hide multiple platform actions. For auditability, the semantic action must be recorded explicitly.

Y.3. Proof Store

Definition 5.10 (Proof Store). A *Proof Store* is a repository for proof objects, certificate records, formal fragments, Bridge Programs, Validity Map entries, Certificate DAG entries, artifact references, ledger records, diagnostic records, and associated metadata.

Definition 5.11 (Proof Store entry). A *Proof Store entry* is a stored item in the Proof Store. It must include:

- (i) entry identifier;

- (ii) object type;
- (iii) content or content reference;
- (iv) status label;
- (v) version identifier;
- (vi) provenance metadata;
- (vii) dependency references, if any;
- (viii) artifact references;
- (ix) audit metadata;
- (x) authority and access metadata.

Definition 5.12 (Stored object). A *stored object* is any object contained in or referenced by the Proof Store.

Definition 5.13 (Stored Core-facing object). A *stored Core-facing object* is a stored object representing or referencing Core-relevant verifier-side material, such as:

$$\text{Eval}_{i,W,\tau}(F), \quad C_{\tau,W}F, \quad \phi_{i,W,\tau}, \quad B\text{-Gate}^+, \quad (\mu_{\text{Collapse}}, u_{\text{Collapse}}), \quad \Sigma\delta < \text{gap}_{\tau}.$$

Definition 5.14 (Verified Core-facing object). A *verified Core-facing object* is a stored Core-facing object whose relevant Core verifier-side conditions have been checked and whose verification record is stored and audit-readable.

Proposition 5.15 (Stored object is not certified by storage). A *stored object* is not certified merely by being stored in the Proof Store.

Proof. Storage records availability and identity. Certification requires verifier-side conditions, proof data, diagnostics, ledgers, gate verdicts, and audit-readable manifests. Therefore storage alone does not certify an object. $\square$

Proposition 5.16 (Stored Core-facing object is not verified by storage). A *stored Core-facing object* is not Core-verified merely by being stored.

Proof. A stored Core-facing object may represent a repaired evaluation record, representative record, tower artifact, diagnostic, or ledger condition. Core verification requires checking the corresponding verifier-side criteria. Storage records the object but does not perform or imply that check. $\square$

Remark 5.17 (Storage supports, but does not replace, verification). The Proof Store is essential for reproducibility and audit. Its function is to preserve and organize evidence, not to create validity automatically.

Y.4. Object classes

Definition 5.18 (Proof Store object class). A *Proof Store object class* is a declared type of object stored in the Proof Store. Standard classes include:

- (i) search artifact;
- (ii) candidate;

- (iii) bridge draft;
- (iv) bridge candidate;
- (v) Bridge Program;
- (vi) bridge theorem;
- (vii) formal fragment;
- (viii) verified formal fragment;
- (ix) certificate candidate;
- (x) Core certificate;
- (xi) Core-facing object;
- (xii) verified Core-facing object;
- (xiii) interface theorem;
- (xiv) proof-first record;
- (xv) proof object;
- (xvi) theorem-level proof object;
- (xvii) ledger record;
- (xviii) diagnostic record;
- (xix) artifact reference;
- (xx) interpretation theorem;
- (xxi) non-claim;
- (xxii) rejected object;
- (xxiii) deprecated object;
- (xxiv) superseded object.

Definition 5.19 (Object status). An *object status* is a validity or lifecycle label attached to a Proof Store object. It must be compatible with the status taxonomy of Appendices U, V, W, and X.

Definition 5.20 (Status authority). A *status authority* is the declared agent, tool, procedure, or governance rule authorized to assign or update a particular object status.

Proposition 5.21 (Object class does not imply validity). *The object class of a stored item does not by itself imply validity.*

Proof. An object class describes what kind of object is stored. Validity depends on proof, verification, status, discharge, and audit records. Therefore class and validity are distinct. $\square$

Remark 5.22 (Class/status separation). A formal fragment may be unverified or verified. A Core-facing object may be stored but unchecked. A certificate candidate may pass or fail. A bridge theorem may later be superseded. Thus object class and object status must be tracked separately.

Y.5. Claim Registry

Definition 5.23 (Claim Registry). A *Claim Registry* is a platform component that stores mathematical, operational, interface, bridge, platform, and non-claim statements together with status labels, dependencies, proof records, and artifact references.

Definition 5.24 (Registered claim). A *registered claim* is a claim recorded in the Claim Registry. It must include:

- (i) claim identifier;
- (ii) claim text;
- (iii) claim type;
- (iv) status label;
- (v) dependencies;
- (vi) supporting artifacts;
- (vii) responsible agents or roles;
- (viii) verification or rejection record, if any;
- (ix) demotion or supersession record, if any.

Definition 5.25 (Claim type). A *claim type* may be:

- (i) Core theorem;
- (ii) Core lemma;
- (iii) Core certificate claim;
- (iv) operational policy;
- (v) interface theorem;
- (vi) bridge draft;
- (vii) bridge candidate;
- (viii) bridge theorem;
- (ix) [Spec];
- (x) search artifact;
- (xi) platform claim;
- (xii) non-claim;
- (xiii) rejected claim.

Proposition 5.26 (Registration is not validation). A *registered claim* is not validated merely by being registered.

Proof. Registration records the claim and its metadata. Validation requires proof or verification under the declared status criteria. Therefore registration is not validation. $\square$

Remark 5.27 (Registry prevents orphan claims). The Claim Registry helps prevent unsupported or forgotten claims by requiring status labels, dependencies, and artifact references.

Y.6. Artifact Store

Definition 5.28 (Artifact Store). An *Artifact Store* is a repository for files, datasets, code, logs, formal proof scripts, numerical outputs, barcodes, diagrams, manifests, ledger files, diagnostic files, replay records, and other artifacts referenced by proof objects or certificates.

Definition 5.29 (Artifact entry). An *artifact entry* contains:

- (i) artifact identifier;
- (ii) artifact type;
- (iii) content address or location;
- (iv) hash or checksum, if used;
- (v) version identifier;
- (vi) creator or source;
- (vii) associated proof-store entries;
- (viii) supported claim, node, or edge references;
- (ix) license or access constraints, if relevant;
- (x) audit notes.

Definition 5.30 (Artifact binding). An *artifact binding* is a declared relation between a proof-store object and an artifact entry.

Definition 5.31 (Artifact adequacy). *Artifact adequacy* is the property that an artifact supports the specific claim, computation, proof, diagnostic, ledger, or certificate condition for which it is cited.

Proposition 5.32 (Artifact availability is not artifact adequacy). *The availability of an artifact does not imply that it adequately supports the claim that cites it.*

Proof. Availability means the artifact can be retrieved. Adequacy means it supports the relevant claim, proof, computation, diagnostic, ledger, or certificate condition. These are distinct properties. $\square$

Remark 5.33 (Artifact adequacy is audit matter). Whether an artifact adequately supports a claim must be checked during verification or audit.

Y.7. Versioning and immutability

Definition 5.34 (Version identifier). A *version identifier* is a label or content-derived identifier distinguishing one version of an object or artifact from another.

Definition 5.35 (Immutable object). An *immutable object* is an object whose proof-relevant content is not changed after storage. Any modification creates a new version.

Definition 5.36 (Mutable record). A *mutable record* is a platform record whose metadata, status, or pointers may change over time while preserving history.

Definition 5.37 (Version lineage). A *version lineage* is the recorded history of object versions and their relationships.

Definition 5.38 (Status inheritance). *Status inheritance* is the attempted transfer of a status label from one version of an object to another.

Proposition 5.39 (Version change may require re-verification). *If a stored object's proof-relevant content changes, any verification or audit status depending on the old content does not automatically transfer to the new version.*

Proof. Verification applies to the object content that was checked. A changed version may have different definitions, proofs, dependencies, or artifacts. Therefore status transfer requires justification or re-verification. $\square$

Proposition 5.40 (Status inheritance requires justification). *Status inheritance from one version to another is admissible only if a preservation argument, equivalence check, or re-verification record justifies it.*

Proof. A status label is attached to a verified or reviewed object under declared conditions. If the object changes, those conditions may no longer hold. Thus inheritance requires explicit justification. $\square$

Remark 5.41 (Immutable content, mutable metadata). A robust Proof Store should treat proof-relevant content as immutable while allowing metadata and status records to evolve with full history.

Y.8. Hashing and content identity

Definition 5.42 (Content hash). A *content hash* is a deterministic identifier derived from the content of an object or artifact.

Definition 5.43 (Hash binding). A *hash binding* is a relation between an object identifier and a content hash.

Definition 5.44 (Hash mismatch). A *hash mismatch* occurs when retrieved content does not match the declared hash.

Definition 5.45 (Content identity). *Content identity* means that the retrieved content agrees with the declared stored content according to the platform identity policy.

Proposition 5.46 (Hash match proves identity, not validity). *A content hash match supports content identity, but it does not prove mathematical validity.*

Proof. A hash can show that the retrieved content is the same as the stored or referenced content. It does not establish that the content is true, verified, sufficient, or theorem-level. $\square$

Proposition 5.47 (Hash mismatch invalidates artifact binding until resolved). *A hash mismatch invalidates the affected artifact binding until the mismatch is resolved.*

Proof. If the content no longer matches the declared identity, the proof-store object may be referencing the wrong artifact. The binding is therefore not audit-stable until resolved. $\square$

Remark 5.48 (Hashing controls drift). Hashing is a practical mechanism for controlling artifact drift, but it must be paired with adequacy checks.

Y.9. Proof object lifecycle

Definition 5.49 (Proof object lifecycle). The *proof object lifecycle* is the sequence of states through which a proof-related object may pass:

created $\longrightarrow$ stored $\longrightarrow$ labeled $\longrightarrow$ linked $\longrightarrow$ verified $\longrightarrow$ audited $\longrightarrow$ promoted or rejected.

Definition 5.50 (Lifecycle event). A *lifecycle event* is a recorded state change or action in the proof object lifecycle.

Definition 5.51 (Lifecycle log). A *lifecycle log* is an ordered record of lifecycle events for a stored object.

Definition 5.52 (Lifecycle authority). A *lifecycle authority* is the agent, tool, or governance rule authorized to perform a lifecycle event.

Proposition 5.53 (Lifecycle progress is not validity monotonicity). *Progress through the proof object lifecycle does not monotonically imply increasing mathematical validity.*

Proof. An object may be stored, linked, and audited but later rejected due to a discovered gap. Lifecycle progress records workflow state, not mathematical truth. $\square$

Remark 5.54 (Lifecycle logs support accountability). Lifecycle logs support accountability by showing when and why status changes occurred.

Y.10. Promotion, demotion, and rejection

Definition 5.55 (Promotion event). A *promotion event* is a recorded status change to a stronger status, such as from candidate to verified fragment, from bridge candidate to bridge theorem, or from Core certificate candidate to Core verified.

Definition 5.56 (Demotion event). A *demotion event* is a recorded status change to a weaker status, such as from verified to blocked, superseded, rejected, deprecated, or incomplete.

Definition 5.57 (Rejection event). A *rejection event* is a recorded status change indicating that an object or claim has failed the relevant criteria.

Definition 5.58 (Promotion evidence). *Promotion evidence* is the proof, verification record, import record, audit record, discharge certificate, or authority record required to justify promotion.

Proposition 5.59 (Promotion requires recorded evidence). *A promotion event is admissible only if the required evidence for the stronger status is recorded.*

Proof. Promotion changes how an object may be used in proof planning, certification, and theorem-level claims. Without evidence, promotion would enable claim inflation. Therefore evidence is required. $\square$

Proposition 5.60 (Demotion preserves audit integrity). *Demotion preserves audit integrity by preventing known-defective or unsupported objects from retaining stronger statuses.*

Proof. If a defect is discovered, the previous status may no longer be justified. Demotion records this change and prevents continued misuse. $\square$

Remark 5.61 (Rejection is useful information). Rejected objects should usually remain stored with their rejection reason. They prevent repeated work and help map failure regions.

Y.11. AI output management

Definition 5.62 (AI output record). An *AI output record* is a Proof Store or Artifact Store entry recording an AI-generated output and associated metadata.

Definition 5.63 (AI output metadata). AI output metadata may include:

- (i) model or agent identifier;
- (ii) prompt or task description;
- (iii) output content;
- (iv) confidence or ranking, if supplied;
- (v) output status label;
- (vi) human review status;
- (vii) verification status;
- (viii) artifact references;
- (ix) supersession or rejection history, if any.

Definition 5.64 (AI-generated Core-facing record). An *AI-generated Core-facing record* is an AI output claiming to produce or describe Core-facing material, such as:

$$\text{Eval}_{i,W,\tau}(F), \quad C_{\tau,W}F, \quad \phi_{i,W,\tau}, \quad (\mu_{\text{Collapse}}, u_{\text{Collapse}}), \quad \Sigma\delta < \text{gap}_{\tau}.$$

Proposition 5.65 (AI output must be status-labeled). *AI output used in the AK workflow must be assigned an explicit status label.*

Proof. By Appendix U, unlabeled agent output is non-verdict by default. To use AI output responsibly in workflow management, its status must be declared. $\square$

Proposition 5.66 (AI storage does not validate AI output). *Storing AI output in the Proof Store or Artifact Store does not validate it.*

Proof. Storage preserves the output and metadata. Validation requires proof, verification, or audit appropriate to the output's intended status. $\square$

Proposition 5.67 (AI-generated Core-facing records require Core verification). *AI-generated Core-facing records cannot support Core certification unless checked under the declared Core verifier-side protocol.*

Proof. AI generation is proposer-side production. Core certification requires verifier-side checks. Therefore AI-generated Core-facing records remain candidates until verified. $\square$

Remark 5.68 (AI outputs are traceable candidates). AI outputs should be treated as traceable candidates, drafts, summaries, bridge sketches, or search artifacts unless and until verified.

Y.12. Verification records

Definition 5.69 (Verification record). A *verification record* is a Proof Store entry recording the result of a verification action. It must include:

- (i) verified object identifier;
- (ii) verification criteria;
- (iii) verification authority;
- (iv) verification result;
- (v) environment or configuration;
- (vi) artifact references;
- (vii) timestamp or version reference;
- (viii) failure log, if verification failed;
- (ix) scope limitations, if any.

Definition 5.70 (Verification result). A *verification result* is one of:

- (i) pass;
- (ii) fail;
- (iii) inconclusive;
- (iv) not applicable;
- (v) blocked;
- (vi) superseded.

Definition 5.71 (Core verification record). A *Core verification record* is a verification record for a Core-facing object or Core certificate condition. It must specify which Core condition was checked, such as repaired evaluation, admissible representative, eligibility, tower artifact, diagnostic, gate verdict, or ledger budget.

Proposition 5.72 (Verification result is scoped to criteria). A *verification result establishes only what was checked under the declared criteria*.

Proof. Verification criteria define the property being checked. A pass under one criterion does not imply pass under another. Thus the result is scoped. $\square$

Remark 5.73 (Verification record must include environment). For computational or formal verification, the environment and configuration may affect reproducibility. They should be recorded.

Y.13. Audit records

Definition 5.74 (Audit record). An *audit record* is a Proof Store entry recording whether a proof object, certificate, artifact set, platform record, map, DAG, or lifecycle history is independently reconstructible.

Definition 5.75 (Audit result). An *audit result* is one of:

- (i) audit-pass;
- (ii) audit-fail;
- (iii) audit-incomplete;
- (iv) audit-blocked;
- (v) audit-superseded.

Definition 5.76 (Audit scope). The *audit scope* specifies which objects, artifacts, dependencies, ledgers, diagnostics, maps, DAGs, execution records, and proof-store entries were checked.

Proposition 5.77 (Audit pass is not mathematical proof by itself). An *audit-pass result* shows reconstructibility within the declared audit scope, but does not by itself prove the underlying mathematical claim.

Proof. Audit checks reconstructibility and record completeness. Mathematical proof requires the relevant logical or verifier-side conditions to be satisfied. Thus audit pass and proof are distinct. $\square$

Remark 5.78 (Audit is necessary but not sufficient). For high-integrity proof infrastructure, auditability is necessary. It is not sufficient unless the audited proof content is also valid.

Y.14. Proof Store queries

Definition 5.79 (Proof Store query). A *Proof Store query* is a request to retrieve, filter, compare, rank, or inspect stored objects.

Definition 5.80 (Query result). A *query result* is the list, set, graph, table, or summary returned by a Proof Store query.

Definition 5.81 (Status-filtered query). A *status-filtered query* retrieves objects satisfying declared status constraints.

Definition 5.82 (Dependency query). A *dependency query* retrieves dependency relations, map edges, DAG edges, proof-store links, or artifact bindings relevant to one or more objects.

Proposition 5.83 (Query result is retrieval, not verification). A *Proof Store query result* does not verify the returned objects.

Proof. A query retrieves or organizes existing records. Verification requires checking declared criteria. Retrieval is not checking. $\square$

Remark 5.84 (Search bias). Query ranking may bias what users inspect. It should not bias what is accepted as valid.

Y.15. Platform security and access

Definition 5.85 (Access policy). An *access policy* specifies who or what may read, write, modify, verify, audit, promote, demote, reject, supersede, or delete platform objects.

Definition 5.86 (Write authority). A *write authority* is permission to create or modify Proof Store or Artifact Store entries.

Definition 5.87 (Promotion authority). A *promotion authority* is permission to change an object's status to a stronger status.

Definition 5.88 (Demotion authority). A *demotion authority* is permission to change an object's status to a weaker or safer status.

Definition 5.89 (Verification authority record). A *verification authority record* identifies the agent, tool, or procedure authorized to perform a verification action and its authority scope.

Proposition 5.90 (Write authority is not promotion authority). *Write authority does not imply promotion authority.*

Proof. Writing or modifying records is operational access. Promotion changes validity status. These permissions have different risk profiles and must be separately controlled. $\square$

Remark 5.91 (Access control protects claim status). Access control is not merely cybersecurity. It protects the integrity of mathematical status labels and proof records.

Y.16. Platform manifest entries

Definition 5.92 (Platform manifest entry). A *platform manifest entry* records:

- (i) platform component;
- (ii) action performed;
- (iii) agent or process identifier;
- (iv) input objects;
- (v) output objects;
- (vi) object classes;
- (vii) object statuses;
- (viii) version identifiers;
- (ix) hashes or content identities, if used;
- (x) verification or audit records, if any;
- (xi) access and authority scope;
- (xii) artifact references;
- (xiii) lifecycle event identifier.

Definition 5.93 (Proof Store manifest). A *Proof Store manifest* is a collection of platform manifest entries sufficient to reconstruct the lifecycle of a proof object, certificate record, map node, DAG node, bridge program, or stored artifact.

Proposition 5.94 (Platform manifests support reproducibility). *Platform manifests support reproducibility by recording actions, versions, identities, statuses, authorities, and artifact references.*

Proof. Reproducibility requires reconstructing not only the content but also the workflow and status history. Platform manifests record these elements. $\square$

Remark 5.95 (Manifest still needs execution schema). The execution-level schema for replay, environment capture, run configuration, and computational reproducibility is specified in Appendix Z.

Y.17. Proof Store and DAG synchronization

Definition 5.96 (Map-store synchronization). *Map-store synchronization* is the process of keeping Validity Map nodes and Proof Store entries consistent in identifiers, statuses, artifacts, and lifecycle history.

Definition 5.97 (DAG-store synchronization). *DAG-store synchronization* is the process of keeping Certificate DAG nodes, edges, discharge records, and Proof Store entries mutually consistent.

Definition 5.98 (Synchronization failure). A *synchronization failure* occurs when a Proof Store entry, Validity Map node, Certificate DAG node, artifact binding, or status record becomes inconsistent with its counterpart.

Proposition 5.99 (Synchronization failure requires review). *A synchronization failure affecting a necessary proof dependency requires review and may require demotion or quarantine.*

Proof. If the Proof Store and map/DAG records disagree, an auditor cannot reliably reconstruct dependency status. A necessary dependency with inconsistent records is not audit-stable until reviewed. $\square$

Remark 5.100 (Synchronization is governance, not proof). Synchronization keeps records coherent. It does not by itself verify the mathematical content of those records.

Y.18. Summary of AI Platform and Proof Store

Theorem 5.101 (Appendix Y platform-store package). *Within the Search / Platform layer, the following package is available.*

- (i) *The AI Platform supports search, storage, verification, audit, replay, and reproducibility workflows.*
- (ii) *Platform actions are scoped.*
- (iii) *A Proof Store stores proof-related objects but does not certify them by storage.*
- (iv) *Stored Core-facing objects are not Core-verified merely by storage.*
- (v) *Object class and object status must be separated.*
- (vi) *Registration is not validation.*
- (vii) *Artifact availability is not artifact adequacy.*
- (viii) *Version changes may require re-verification.*

- (ix) *Status inheritance requires justification.*
- (x) *Hash matches support identity, not validity.*
- (xi) *Hash mismatches invalidate affected artifact bindings until resolved.*
- (xii) *Lifecycle progress is not validity monotonicity.*
- (xiii) *Promotion requires recorded evidence.*
- (xiv) *AI output must be status-labeled, and AI storage does not validate it.*
- (xv) *AI-generated Core-facing records require Core verification.*
- (xvi) *Verification results are scoped to declared criteria.*
- (xvii) *Audit pass is not mathematical proof by itself.*
- (xviii) *Query results are retrieval, not verification.*
- (xix) *Write authority is not promotion authority.*
- (xx) *Platform manifests support reproducibility but do not themselves prove mathematical claims.*
- (xxi) *Map-store and DAG-store synchronization failures require review when they affect necessary dependencies.*

Proof. Items (i)–(ii) follow from the AI Platform and platform action definitions and Proposition 5.8. Item (iii) is Proposition 5.15. Item (iv) is Proposition 5.16. Item (v) follows from object class and status definitions and Proposition 5.21. Item (vi) is Proposition 5.26. Item (vii) is Proposition 5.32. Item (viii) is Proposition 5.39. Item (ix) is Proposition 5.40. Items (x)–(xi) are Propositions 5.46 and 5.47. Item (xii) is Proposition 5.53. Item (xiii) is Proposition 5.59. Item (xiv) follows from Propositions 5.65 and 5.66. Item (xv) is Proposition 5.67. Item (xvi) is Proposition 5.72. Item (xvii) is Proposition 5.77. Item (xviii) is Proposition 5.83. Item (xix) is Proposition 5.90. Item (xx) follows from Proposition 5.94. Item (xxi) is Proposition 5.99. □

Y.19. Explicit exclusions

The following are not asserted in Appendix Y.

- No stored object is certified by storage alone.
- No stored Core-facing object is Core-verified by storage alone.
- No registered claim is validated by registration alone.
- No artifact is adequate merely because it is available.
- No hash match proves mathematical validity.
- No version tag proves theorem status.
- No lifecycle progress proves increasing validity.
- No AI output is validated by being stored.

- No AI confidence value is treated as proof.
- No AI-generated Core-facing record is accepted without Core verification.
- No query result is treated as verification.
- No audit pass is treated as mathematical proof by itself.
- No write authority is treated as promotion authority.
- No promotion is allowed without recorded evidence.
- No status inheritance is allowed without justification.
- No Proof Store entry replaces B-Gate⁺.
- No Proof Store entry replaces the repaired topological condition

$$\text{Eval}_{1,W,\tau}(F) = 0.$$

- No Proof Store entry replaces admissible representative records

$$C_{\tau,W}F.$$

- No Proof Store entry replaces monitored Type IV diagnostics.
- No Proof Store entry replaces tower comparison artifacts

$$\phi_{i,W,\tau}.$$

- No Proof Store entry replaces the budget condition

$$\Sigma\delta < \text{gap}_{\tau}.$$

- No platform storage event turns a Bridge Program into a bridge theorem.
- No platform record turns a Validity Map into a theorem.
- No map-store or DAG-store synchronization event proves the synchronized mathematical content.

Y.20. Provenance and status

The material in this appendix depends on:

- (i) the Agent Semantics of Appendix U;
- (ii) the Hunter / Mapper / Lifter protocols of Appendix V;
- (iii) the Bridge Programs of Appendix W;
- (iv) the Validity Map and Global Certificate DAG of Appendix X;
- (v) the repaired evaluation discipline of Appendix A;
- (vi) the admissible representative discipline of Appendix B;

- (vii) the eligible realization regime of Appendix C;
- (viii) the monitored Type IV diagnostic and tower artifact discipline of Appendix D;
- (ix) the Core sovereignty doctrine of Chapter 6;
- (x) the non-claim discipline of Chapter 7;
- (xi) the manifest schema of Appendix G;
- (xii) the ledger catalog of Appendix M.

Its role is to provide the storage, registry, lifecycle, synchronization, and platform infrastructure required for auditable proof management without confusing storage with certification.

Status labels for Appendix Y. *Search / Platform definitions:* Definitions [5.4](#), [5.5](#), [5.6](#), [5.7](#), [5.10](#), [5.11](#), [5.12](#), [5.13](#), [5.14](#), [5.18](#), [5.19](#), [5.20](#), [5.23](#), [5.24](#), [5.25](#), [5.28](#), [5.29](#), [5.30](#), [5.31](#), [5.34](#), [5.35](#), [5.36](#), [5.37](#), [5.38](#), [5.42](#), [5.43](#), [5.44](#), [5.45](#), [5.49](#), [5.50](#), [5.51](#), [5.52](#), [5.55](#), [5.56](#), [5.57](#), [5.58](#), [5.62](#), [5.63](#), [5.64](#), [5.69](#), [5.70](#), [5.71](#), [5.74](#), [5.75](#), [5.76](#), [5.79](#), [5.80](#), [5.81](#), [5.82](#), [5.85](#), [5.86](#), [5.87](#), [5.88](#), [5.89](#), [5.92](#), [5.93](#), [5.96](#), [5.97](#), [5.98](#).

Search / Platform propositions/theorems: Propositions [5.8](#), [5.15](#), [5.16](#), [5.21](#), [5.26](#), [5.32](#), [5.39](#), [5.40](#), [5.46](#), [5.47](#), [5.53](#), [5.59](#), [5.60](#), [5.65](#), [5.66](#), [5.67](#), [5.72](#), [5.77](#), [5.83](#), [5.90](#), [5.94](#), [5.99](#), Theorem [5.101](#).

Operational cautions: Remarks [5.1](#), [5.2](#), [5.3](#), [5.9](#), [5.17](#), [5.22](#), [5.27](#), [5.33](#), [5.41](#), [5.48](#), [5.54](#), [5.61](#), [5.68](#), [5.73](#), [5.78](#), [5.84](#), [5.91](#), [5.95](#), [5.100](#).

Explicit exclusions: Section Y.19.

6 Appendix Z: Execution and Reproducibility Schema

Z.1. Scope and reproducibility status

This appendix defines the execution and reproducibility schema for the AK framework. It belongs to the Search / Platform Appendices. It does not enlarge the Core mathematical package of Chapters 1–8.

The central rule of this appendix is:

Reproducibility supports audit; it is not proof by itself.

A reproducible execution record may allow another auditor to reconstruct a run, recompute artifacts, verify hashes, inspect diagnostics, replay ledger aggregation, replay gate checks, and inspect platform actions. However, reproducibility alone does not establish mathematical validity.

In particular, no execution record, replay log, run configuration, environment hash, container image, output file, reproducibility report, or replay-pass result may replace:

$$B\text{-Gate}^+, \quad \text{Eval}_{1,W,\tau}(F) = 0, \quad C_{\tau,W}F, \quad \phi_{i,W,\tau}, \quad (\mu_{\text{Collapse}}, u_{\text{Collapse}}), \quad \Sigma\delta < \text{gap}_{\tau}.$$

Remark 6.1 (Search / Platform status). This appendix specifies how runs, artifacts, ledgers, manifests, diagnostics, gate records, and environments should be recorded so that results are reconstructible. It does not create new theorems.

Remark 6.2 (Relation to Appendix Y). Appendix Y defines the AI Platform and Proof Store. The present appendix defines the execution-level schema by which platform entries become replayable and audit-readable.

Remark 6.3 (Final platform appendix). This is the final Search / Platform Appendix. Its role is to close the operational infrastructure around the AK Core while preserving the distinction between execution, reproducibility, auditability, Core certification, and theorem-level proof.

Z.2. Execution run

Definition 6.4 (Execution run). An *execution run* is a recorded instance of an AK workflow, computation, verification, audit, replay, search, lifting, diagnostic calculation, ledger calculation, gate check, or certificate assembly.

Definition 6.5 (Run identifier). A *run identifier* is a unique label assigned to an execution run.

Definition 6.6 (Run scope). The *run scope* specifies what the execution run is intended to do. Examples include:

(i) compute persistence artifacts;

(ii) compute repaired evaluations

$$\text{Eval}_{i,w,\tau}(F);$$

(iii) construct or check admissible representative records

$$C_{\tau,w}F;$$

(iv) compute or check tower comparison artifacts

$$\phi_{i,w,\tau};$$

(v) compute monitored Type IV diagnostics;

(vi) evaluate ledger budgets;

(vii) check B-Gate⁺;

(viii) replay a certificate;

(ix) verify a formal fragment;

(x) assemble a Certificate DAG;

(xi) reproduce a search result;

(xii) audit artifact completeness.

Definition 6.7 (Run authority). A *run authority* is the agent, tool, platform component, or verification procedure authorized to execute or validate a run within its declared scope.

Proposition 6.8 (Run scope limits interpretation). An *execution run* establishes only what lies within its declared run scope.

Proof. A run is configured to perform a declared task. Outputs outside that task are not checked by the run unless explicitly included. Therefore interpretation is limited by the run scope. $\square$

Remark 6.9 (Run success is scoped). A successful run may mean that a script executed, a file was generated, a diagnostic was computed, a gate clause was checked, or a proof assistant accepted a formal fragment. These meanings are distinct and must be recorded separately.

Z.3. Execution schema

Definition 6.10 (Execution schema). An *execution schema* is a structured specification of the inputs, parameters, environment, tools, artifacts, commands, outputs, logs, and verification criteria of an execution run.

Definition 6.11 (run.yaml). A run.yaml file is a human-readable execution schema file describing an AK execution run. It should contain:

- (i) run identifier;
- (ii) run scope;
- (iii) run authority;
- (iv) input artifacts;
- (v) output artifacts;
- (vi) commands or workflow steps;
- (vii) environment references;
- (viii) parameter values;
- (ix) random seeds, if applicable;
- (x) repaired evaluation configuration, if applicable;
- (xi) representative configuration, if applicable;
- (xii) tower artifact configuration, if applicable;
- (xiii) ledger configuration;
- (xiv) diagnostic configuration;
- (xv) gate configuration;
- (xvi) verification criteria;
- (xvii) expected outputs;
- (xviii) artifact hashes or hash policy;
- (xix) replay instructions.

Definition 6.12 (Execution parameter). An *execution parameter* is a value controlling an execution run. Examples include:

τ , W , i , β , $M(\tau)$, s , grid size, random seed.

Definition 6.13 (Parameter binding). A *parameter binding* is a recorded assignment of a value to an execution parameter within a run.

Proposition 6.14 (Execution schema is required for replayability). *A run is replayable only if its execution schema records sufficient information to reconstruct the run.*

Proof. Replay requires reconstructing inputs, parameters, commands, environment, and expected outputs. If these data are missing, the run cannot be reproduced. Therefore a sufficient execution schema is required. $\square$

Remark 6.15 (Schema completeness is task-relative). A schema sufficient for recomputing barcodes may not be sufficient for replaying a proof assistant verification or a Certificate DAG assembly. Schema completeness depends on the run scope.

Z.4. Artifact manifest

Definition 6.16 (Artifact manifest). An *artifact manifest* is a structured list of all artifacts used or produced by an execution run.

Definition 6.17 (Input artifact). An *input artifact* is an artifact required by an execution run.

Definition 6.18 (Output artifact). An *output artifact* is an artifact produced by an execution run.

Definition 6.19 (Intermediate artifact). An *intermediate artifact* is an artifact produced during a run and consumed by later steps of the same or dependent runs.

Definition 6.20 (Artifact manifest entry). An *artifact manifest entry* records:

- (i) artifact identifier;
- (ii) artifact role: input, output, intermediate, log, reference, or replay artifact;
- (iii) artifact type;
- (iv) storage location or content address;
- (v) version identifier;
- (vi) hash or checksum, if used;
- (vii) producing run, if applicable;
- (viii) consuming runs, if known;
- (ix) adequacy notes;
- (x) audit notes.

Proposition 6.21 (Artifact manifest supports audit but does not prove adequacy). *An artifact manifest supports auditability, but it does not by itself prove that the artifacts are adequate for the claims that cite them.*

Proof. The manifest records artifact existence, identity, and role. Adequacy requires checking that each artifact supports the relevant claim, computation, diagnostic, ledger entry, gate clause, or certificate condition. Thus manifest presence and artifact adequacy are distinct. $\square$

Remark 6.22 (Completeness versus adequacy). A manifest may be complete but still inadequate if the listed artifacts do not support the intended claims. Both completeness and adequacy must be considered during audit.

Z.5. Environment capture

Definition 6.23 (Execution environment). An *execution environment* is the collection of software, hardware, operating system, dependencies, libraries, proof assistant versions, computation engines, container images, and configuration settings used in an execution run.

Definition 6.24 (Environment capture). *Environment capture* is the process of recording the execution environment sufficiently to support replay.

Definition 6.25 (Environment identifier). An *environment identifier* is a label, hash, container digest, dependency lockfile, proof assistant environment record, or platform record identifying an execution environment.

Definition 6.26 (Environment compatibility). *Environment compatibility* is a declared relation specifying when a different environment may be accepted as replay-equivalent to the original environment.

Proposition 6.27 (Environment capture is required for computational replay). *Computational replay requires environment capture sufficient for the declared run scope.*

Proof. Different environments may produce different outputs due to dependency versions, numerical libraries, proof assistant versions, or system configuration. Replay requires controlling or recording these dependencies. Therefore environment capture is required. $\square$

Remark 6.28 (Environment equality is stronger than needed in some cases). Exact environment equality may be unnecessary for purely formal or deterministic checks if semantic compatibility is proved. However, absent such a proof, environment capture should be conservative.

Z.6. Determinism and randomness

Definition 6.29 (Deterministic run). A run is *deterministic* if the same inputs, parameters, environment, and commands produce the same outputs under the declared replay equivalence.

Definition 6.30 (Randomized run). A run is *randomized* if it depends on random seeds, sampling, stochastic optimization, randomized search, probabilistic algorithms, or nondeterministic execution.

Definition 6.31 (Random seed policy). A *random seed policy* specifies how random seeds are chosen, recorded, replayed, and varied.

Definition 6.32 (Nondeterminism record). A *nondeterminism record* records sources of nondeterminism in a run, including parallel execution, hardware nondeterminism, randomized algorithms, sampling procedures, and external service calls.

Proposition 6.33 (Randomized runs require seed or distribution records). *A randomized run is reproducible only if the seed, sampling policy, nondeterminism record, or probability distribution is recorded sufficiently for the run scope.*

Proof. Without seed, distribution, or nondeterminism records, the random choices cannot be reconstructed or statistically characterized. Therefore reproducibility requires recording them. $\square$

Remark 6.34 (Statistical reproducibility). For randomized search, exact replay may be less important than statistical reproducibility. If so, the statistical criterion must be declared.

Z.7. Repaired evaluation execution files

Definition 6.35 (eval.json). An eval.json file is a structured record of repaired evaluation computations:

$$\text{Eval}_{i,W,\tau}(F) = T_{\tau}^{\text{bar}}(W_W \mathbf{P}_i(F)).$$

Definition 6.36 (Evaluation execution entry). An *evaluation execution entry* records:

- (i) input object identifier;
- (ii) monitored degree i ;
- (iii) window W ;
- (iv) threshold τ ;
- (v) persistence extraction method;
- (vi) barcode policy;
- (vii) window restriction policy;
- (viii) threshold deletion policy;
- (ix) resulting repaired profile;
- (x) vanishing or nonvanishing result, if checked;
- (xi) artifact references.

Proposition 6.37 (Repaired evaluation replay requires evaluation records). *A repaired evaluation is replayable only if the evaluation execution record contains sufficient data to reconstruct*

$$\text{Eval}_{i,W,\tau}(F).$$

Proof. The repaired evaluation depends on the input object, persistence extraction, degree, window, threshold, barcode policy, and threshold deletion policy. Without these records, an auditor cannot reconstruct the evaluation. $\square$

Remark 6.38 (Evaluation summary is not enough). A statement such as

$$\text{Eval}_{1,W,\tau}(F) = 0$$

is not audit-readable unless the supporting evaluation computation or proof can be reconstructed.

Z.8. Ledger execution files

Definition 6.39 (ledger.json). A ledger.json file is a structured record of ledger components, aggregation semantics, gap values, and pass/fail budget comparisons used in an execution run.

Definition 6.40 (Ledger execution entry). A *ledger execution entry* records:

- (i) ledger semantics;
- (ii) component identifiers;

- (iii) component values or bounds;
- (iv) component sources;
- (v) aggregation law;
- (vi) total budget $\Sigma\delta$;
- (vii) gap value gap_τ ;
- (viii) strict comparison convention;
- (ix) pass/fail result;
- (x) artifact references.

Proposition 6.41 (Ledger replay requires ledger execution records). *A budget comparison is replayable only if the ledger execution record contains sufficient component and aggregation data.*

Proof. Budget replay requires recomputing or checking

$$\Sigma\delta < \text{gap}_\tau.$$

Without component values, aggregation semantics, and gap values, the comparison cannot be reconstructed. $\square$

Remark 6.42 (Ledger file is not budget pass by itself). The existence of ledger.json does not imply budget pass. The recorded comparison must actually satisfy the declared strict inequality.

Z.9. Diagnostic execution files

Definition 6.43 (diagnostics.json). A diagnostics.json file is a structured record of diagnostic computations, including declared tower comparison artifacts, kernel and cokernel defect data, generic fiber dimensions, and Type IV verdicts.

Definition 6.44 (Diagnostic execution entry). *A diagnostic execution entry records:*

- (i) diagnostic type;
- (ii) monitored degree;
- (iii) window;
- (iv) threshold;
- (v) tower comparison artifact

$$\phi_{i,W,\tau};$$

- (vi) source profile

$$\text{ColimProfile}_{i,W,\tau}(F_\bullet);$$

- (vii) target profile

$$\text{ApexProfile}_{i,W,\tau}(F_\infty);$$

- (viii) kernel defect object;

- (ix) cokernel defect object;
- (x) generic fiber dimensions;
- (xi) values of μ_{Collapse} and u_{Collapse} , if applicable;
- (xii) diagnostic verdict;
- (xiii) artifact references.

Proposition 6.45 (Diagnostic replay requires diagnostic execution records). *A monitored Type IV diagnostic is replayable only if the diagnostic execution record contains sufficient data to reconstruct the tower comparison artifact and defect calculations.*

Proof. Type IV diagnostics are defined by kernel and cokernel defect objects of the declared tower comparison artifact

$$\phi_{i,W,\tau}.$$

Replay requires reconstructing this artifact and its defect dimensions. Therefore the diagnostic execution record must include the required data. $\square$

Remark 6.46 (Diagnostic summary is not enough). A line saying

$$(\mu_{\text{Collapse}}, u_{\text{Collapse}}) = (0, 0)$$

is not enough for audit unless the supporting diagnostic computation can be reconstructed.

Z.10. Gate execution files

Definition 6.47 (gate.json). A gate.json file is a structured record of Core gate checks, including B-Gate⁺, admissibility conditions, representative requirements, diagnostics, ledger conditions, and gate verdicts.

Definition 6.48 (Gate execution entry). A *gate execution entry* records:

- (i) gate identifier;
- (ii) gate definition;
- (iii) input object;
- (iv) window and threshold;
- (v) checked clauses;
- (vi) repaired evaluation clause, if used;
- (vii) representative clause, if used;
- (viii) eligibility clause, if used;
- (ix) tower diagnostic clause, if used;
- (x) ledger clause, if used;
- (xi) clause-level results;

- (xii) final gate verdict;
- (xiii) verifying authority;
- (xiv) artifact references.

Proposition 6.49 (Gate verdict requires clause-level support). *A gate verdict is audit-readable only if its required clauses are recorded with sufficient support.*

Proof. A gate verdict is determined by its clauses. Without clause-level records, an auditor cannot reconstruct why the verdict was assigned. Therefore clause-level support is required. $\square$

Remark 6.50 (Gate file is not self-certifying). The existence of `gate.json` does not by itself prove that the gate passed. Its contents must support the verdict.

Z.11. Replay

Definition 6.51 (Replay). A *replay* is an attempt to reproduce an execution run from its recorded schema, environment, inputs, parameters, commands, and artifacts.

Definition 6.52 (Replay result). A *replay result* is one of:

- (i) replay-pass;
- (ii) replay-fail;
- (iii) replay-incomplete;
- (iv) replay-divergent;
- (v) replay-not-applicable;
- (vi) replay-blocked.

Definition 6.53 (Replay equivalence). *Replay equivalence* is a declared relation specifying when replay output is considered equivalent to the original output. It may require exact equality, hash equality, numerical tolerance, proof assistant acceptance, diagnostic equality, ledger equality, gate verdict equality, or semantic equivalence.

Definition 6.54 (Replay witness). A *replay witness* is an artifact, log, hash, proof assistant output, diagnostic output, ledger output, or gate output supporting a replay result.

Proposition 6.55 (Replay pass is not theorem proof by itself). *A replay-pass result shows reproducibility under the declared replay criterion, but it does not by itself prove the underlying mathematical claim.*

Proof. Replay checks whether a run can be reproduced. Mathematical proof requires the relevant verifier-side or logical conditions. Reproduction and proof are distinct. $\square$

Remark 6.56 (Replay criterion must be declared). For numerical runs, exact equality may be too strict or too weak depending on the task. For formal runs, proof assistant acceptance may be appropriate. For Core diagnostics, diagnostic equality or certified semantic equivalence may be required. The replay equivalence criterion must be declared.

Z.12. Reproducibility levels

Definition 6.57 (Reproducibility level). A *reproducibility level* classifies how much of an execution run can be reconstructed.

Definition 6.58 (Level R0). *Level R0* means that the result is stated but no sufficient execution schema or artifacts are available.

Definition 6.59 (Level R1). *Level R1* means that artifacts are referenced, but replay is not possible from the record.

Definition 6.60 (Level R2). *Level R2* means that inputs, parameters, and outputs are recorded, but the environment or commands are incomplete.

Definition 6.61 (Level R3). *Level R3* means that the run can be replayed in a compatible environment with declared tolerances.

Definition 6.62 (Level R4). *Level R4* means that the run can be replayed exactly or content-identically using captured environment, artifacts, commands, and hashes.

Definition 6.63 (Level R5). *Level R5* means that the result is reproducible and independently audited, with all required proof-store, artifact-store, evaluation, ledger, diagnostic, gate, map, DAG, and replay records linked.

Proposition 6.64 (Higher reproducibility level does not imply theorem validity). A *higher reproducibility level improves auditability but does not by itself imply theorem validity*.

Proof. Reproducibility concerns reconstruction of runs and artifacts. Theorem validity concerns the correctness of mathematical claims. A reproducible invalid computation remains invalid. $\square$

Remark 6.65 (Preferred level). For high-stakes Core certificates, Level R5 is preferred. However, even R5 must be paired with valid proof content.

Z.13. Formal verification replay

Definition 6.66 (Formal verification replay). A *formal verification replay* is a replay in which proof assistant code or formal fragments are checked again under a recorded proof assistant environment.

Definition 6.67 (Formal environment). A *formal environment* records:

- (i) proof assistant name and version;
- (ii) libraries and versions;
- (iii) axioms used;
- (iv) definitions imported;
- (v) build commands;
- (vi) compilation options;
- (vii) formal artifact hashes.

Definition 6.68 (Formal replay artifact). A *formal replay artifact* is a proof assistant output, build log, checked theorem object, compiled file, or formal verification report produced during formal replay.

Proposition 6.69 (Formal replay is scoped to formal statements). *Formal verification replay establishes only the checked formal statements in the recorded formal environment.*

Proof. A proof assistant checks formal statements relative to its environment. It does not automatically prove informal interpretations or external problem-specific claims. Those require interpretation theorems or interface bridges. $\square$

Remark 6.70 (Formal replay and interpretation). A formally replayed theorem may still need an interpretation layer to connect it to the intended AK or problem-specific claim.

Z.14. Numerical reproducibility

Definition 6.71 (Numerical reproducibility). *Numerical reproducibility* is the ability to reproduce numerical outputs within declared tolerances, methods, and environments.

Definition 6.72 (Numerical tolerance). A *numerical tolerance* is a declared acceptable discrepancy between original and replayed numerical outputs.

Definition 6.73 (Numerical stability record). A *numerical stability record* documents sensitivity of numerical outputs to grid, time step, precision, algorithmic choices, and perturbations.

Definition 6.74 (Continuum interpretation record). A *continuum interpretation record* documents the bridge, theorem, or [Spec] status by which numerical or discretized outputs are interpreted as statements about a continuum object.

Proposition 6.75 (Numerical reproducibility is not numerical validity). *Numerical reproducibility does not by itself establish numerical validity or mathematical proof.*

Proof. A numerical computation may be reproducible and still approximate the wrong object, use an unstable method, or fail to justify continuum interpretation. Numerical validity requires additional error, stability, and soundness analysis. $\square$

Remark 6.76 (Relation to Appendices I and J). Appendices I and J provide discretization and measurement stability disciplines. This appendix records the execution schema needed to replay such analyses.

Z.15. Reproducibility manifest

Definition 6.77 (Reproducibility manifest). A *reproducibility manifest* is a top-level record connecting run schema, artifact manifest, environment capture, repaired evaluation records, ledger records, diagnostic records, gate records, replay results, audit records, and proof-store links.

Definition 6.78 (Reproducibility manifest entry). A *reproducibility manifest entry* records:

- (i) run identifier;
- (ii) run scope;
- (iii) run.yaml reference;
- (iv) artifact manifest reference;
- (v) environment reference;
- (vi) eval.json reference, if applicable;

- (vii) ledger.json reference, if applicable;
- (viii) diagnostics.json reference, if applicable;
- (ix) gate.json reference, if applicable;
- (x) replay result;
- (xi) reproducibility level;
- (xii) audit result;
- (xiii) proof-store links;
- (xiv) Validity Map or Certificate DAG links, if applicable.

Proposition 6.79 (Reproducibility manifest supports audit). *A reproducibility manifest supports audit by linking the records needed to reconstruct and inspect an execution run.*

Proof. Audit requires access to the run schema, artifacts, environment, repaired evaluations, ledger, diagnostics, gates, replay results, and proof-store records. The reproducibility manifest links these records. $\square$

Remark 6.80 (Manifest is not proof). A reproducibility manifest may make a run audit-readable. It does not by itself establish that the mathematical conclusion is valid.

Z.16. Reproducibility failure modes

Definition 6.81 (Reproducibility failure). *A reproducibility failure occurs when an execution run cannot be reconstructed, replayed, or audited at the claimed reproducibility level.*

Definition 6.82 (Missing artifact failure). *A missing artifact failure occurs when a required artifact is absent or inaccessible.*

Definition 6.83 (Environment failure). *An environment failure occurs when the recorded environment is insufficient, unavailable, or incompatible with replay.*

Definition 6.84 (Parameter failure). *A parameter failure occurs when required execution parameters are missing, ambiguous, or inconsistent.*

Definition 6.85 (Replay divergence). *Replay divergence occurs when replayed outputs fail the declared replay equivalence criterion.*

Definition 6.86 (Manifest inconsistency). *Manifest inconsistency occurs when run schema, artifact manifest, evaluation records, ledger records, diagnostic records, gate records, or proof-store entries disagree.*

Definition 6.87 (Evaluation replay failure). *An evaluation replay failure occurs when*

$$\text{Eval}_{i,w,\tau}(F)$$

cannot be reconstructed or does not satisfy the declared replay equivalence.

Definition 6.88 (Diagnostic replay failure). *A diagnostic replay failure occurs when monitored diagnostic values or tower artifact computations cannot be reconstructed or do not satisfy the declared replay equivalence.*

Proposition 6.89 (Reproducibility failure requires status review). *If reproducibility failure affects a necessary dependency, the corresponding certificate or proof object requires status review and possible demotion.*

Proof. A necessary dependency must be audit-readable and reconstructible at the claimed level. If reproducibility fails, the support for that dependency is weakened. Therefore status review and possible demotion are required. $\square$

Remark 6.90 (Reproducibility failure is not always mathematical failure). A reproducibility failure may indicate missing infrastructure rather than false mathematics. However, until repaired, the affected proof object is not audit-stable.

Z.17. Execution-to-certificate boundary

Definition 6.91 (Execution-to-certificate boundary). The *execution-to-certificate boundary* is the boundary between producing execution artifacts and establishing certificate status.

Definition 6.92 (Execution-supported certificate). An *execution-supported certificate* is a certificate whose computational, diagnostic, ledger, gate, or artifact-dependent components are supported by adequate execution and reproducibility records.

Proposition 6.93 (Execution success does not imply certificate success). *Successful execution of a workflow does not imply that the resulting object is a valid certificate.*

Proof. Execution success may mean that commands ran and outputs were generated. Certificate success requires that the outputs satisfy the Core gate, repaired evaluation, diagnostic, ledger, manifest, and audit conditions. These are distinct. $\square$

Proposition 6.94 (Certificate success requires execution support when computation is used). *If a certificate relies on computed artifacts, then those computations must be supported by adequate execution and reproducibility records.*

Proof. Computed artifacts must be reconstructible and auditable when used as certificate evidence. Execution and reproducibility records provide the required support. $\square$

Remark 6.95 (Boundary discipline). Execution infrastructure should make certificate checking easier. It should never be confused with certificate checking itself.

Z.18. Closure of Part V

Theorem 6.96 (Part V closure principle). *The Search / Platform Appendices U–Z provide agent, search, bridge-program, map, proof-store, and execution infrastructure around the AK Core. They do not enlarge the Core mathematical package, do not replace Core gates, do not replace repaired evaluations, do not replace admissible representatives, do not replace tower comparison artifacts, do not replace monitored Type IV diagnostics, and do not alter the audit budget condition.*

Proof. Appendix U separates proposer-side and verifier-side agent semantics. Appendix V makes search protocols candidate-generating rather than certifying. Appendix W makes Bridge Programs proof-management objects rather than bridge theorems. Appendix X makes Validity Maps and Certificate DAGs navigation and dependency structures rather than proof by existence. Appendix Y makes Proof Store storage non-certifying. The present appendix makes execution and reproducibility audit-supporting rather than proof-generating. Together, these appendices preserve Core sovereignty. $\square$

Remark 6.97 (Meaning of Part V closure). Part V closes the infrastructure layer. It explains how agents, search, bridges, maps, proof stores, and execution records may support proof work without becoming proof merely by being present.

Z.19. Final non-confusion principle

Theorem 6.98 (Final non-confusion principle). *Across the AK framework, the following distinctions are mandatory:*

search eqverification, storage eqcertification, execution eqproof,
reproducibility eqvalidity, Core pass eqexternal theorem without a bridge, [Spec] eqtheorem.

Proof. The first distinction follows from Appendices U and V. The second follows from Appendix Y. The third and fourth follow from the present appendix. The fifth follows from the Problem Interface discipline of Chapter 8 and the soundness-bridge requirements of the Problem Interface Appendices. The sixth follows from the non-claim discipline of Chapter 7. □

Remark 6.99 (Final role of Appendix Z). Appendix Z is the final guard against infrastructure inflation. It ensures that even a fully reproducible, well-stored, agent-assisted, platform-managed, search-discovered, DAG-organized result must still pass the relevant mathematical and verifier-side conditions before it can be treated as proof.

Z.20. Summary of execution and reproducibility schema

Theorem 6.100 (Appendix Z execution-reproducibility package). *Within the Search / Platform layer, the following package is available.*

- (i) *Execution runs must have declared scope.*
- (ii) *Execution schemas such as run.yaml are required for replayability.*
- (iii) *Artifact manifests support audit but do not prove artifact adequacy.*
- (iv) *Environment capture is required for computational replay.*
- (v) *Randomized runs require seed, distribution, or nondeterminism records.*
- (vi) *Repaired evaluation replay requires evaluation execution records.*
- (vii) *Ledger replay requires ledger execution records.*
- (viii) *Diagnostic replay requires diagnostic execution records based on declared tower comparison artifacts.*
- (ix) *Gate verdicts require clause-level support.*
- (x) *Replay pass is not theorem proof by itself.*
- (xi) *Higher reproducibility level improves auditability but does not imply theorem validity.*
- (xii) *Formal verification replay is scoped to formal statements.*
- (xiii) *Numerical reproducibility is not numerical validity.*

- (xiv) *Reproducibility manifests support audit but do not prove mathematical claims.*
- (xv) *Reproducibility failure requires status review when necessary dependencies are affected.*
- (xvi) *Execution success does not imply certificate success.*
- (xvii) *Certificates relying on computed artifacts require adequate execution support.*
- (xviii) *Part V infrastructure does not enlarge the AK Core.*
- (xix) *The final non-confusion principle separates search, storage, execution, reproducibility, verification, Core pass, external theorem status, and [Spec] status.*

Proof. Item (i) follows from the definitions of execution run and run scope. Item (ii) is Proposition 6.14. Item (iii) is Proposition 6.21. Item (iv) is Proposition 6.27. Item (v) is Proposition 6.33. Item (vi) is Proposition 6.37. Item (vii) is Proposition 6.41. Item (viii) is Proposition 6.45. Item (ix) is Proposition 6.49. Item (x) is Proposition 6.55. Item (xi) is Proposition 6.64. Item (xii) is Proposition 6.69. Item (xiii) is Proposition 6.75. Item (xiv) follows from Proposition 6.79 and Remark 6.80. Item (xv) is Proposition 6.89. Item (xvi) is Proposition 6.93. Item (xvii) is Proposition 6.94. Item (xviii) is Theorem 6.96. Item (xix) is Theorem 6.98. $\square$

Z.21. Explicit exclusions

The following are not asserted in Appendix Z.

- No execution run is treated as proof by itself.
- No run.yaml file is treated as certificate by itself.
- No artifact manifest proves artifact adequacy.
- No environment hash proves mathematical validity.
- No random seed proves correctness.
- No eval.json file implies repaired evaluation correctness by existence alone.
- No ledger.json file implies budget pass by existence alone.
- No diagnostics.json file implies Type IV cleanliness by existence alone.
- No gate.json file implies gate pass by existence alone.
- No replay-pass result proves a theorem by itself.
- No high reproducibility level implies theorem validity.
- No formal replay proves informal interpretation without an interpretation theorem.
- No numerical reproducibility proves continuum validity.
- No reproducibility manifest proves a mathematical claim by itself.
- No execution success implies certificate success.

- No reproducibility infrastructure replaces B-Gate⁺.
- No reproducibility infrastructure replaces the repaired topological condition

$$\text{Eval}_{1,W,\tau}(F) = 0.$$

- No reproducibility infrastructure replaces admissible representative records

$$C_{\tau,W}F.$$

- No reproducibility infrastructure replaces monitored Type IV diagnostics.
- No reproducibility infrastructure replaces tower comparison artifacts

$$\phi_{i,W,\tau}.$$

- No reproducibility infrastructure replaces the budget condition

$$\Sigma\delta < \text{gap}_{\tau}.$$

- No Search / Platform appendix upgrades a [Spec] statement into a theorem.
- No Search / Platform appendix converts Core pass into an external theorem without a proved bridge.

Z.22. Provenance and status

The material in this appendix depends on:

- (i) the Agent Semantics of Appendix U;
- (ii) the Hunter / Mapper / Lifter protocols of Appendix V;
- (iii) the Bridge Programs of Appendix W;
- (iv) the Validity Map and Global Certificate DAG of Appendix X;
- (v) the AI Platform and Proof Store of Appendix Y;
- (vi) the repaired evaluation discipline of Appendix A;
- (vii) the admissible representative discipline of Appendix B;
- (viii) the monitored Type IV diagnostic and tower artifact discipline of Appendix D;
- (ix) the Minimal Manifest discipline of Appendix G;
- (x) the ledger catalog of Appendix M;
- (xi) the Core sovereignty doctrine of Chapter 6;
- (xii) the non-claim discipline of Chapter 7;
- (xiii) the interface protocol of Chapter 8.

Its role is to close the Search / Platform layer by specifying how execution records, artifacts, environments, repaired evaluations, ledgers, diagnostics, gates, and replay results are made reproducible without being mistaken for proofs.

Status labels for Appendix Z. *Search / Platform definitions:* Definitions [6.4](#), [6.5](#), [6.6](#), [6.7](#), [6.10](#), [6.11](#), [6.12](#), [6.13](#), [6.16](#), [6.17](#), [6.18](#), [6.19](#), [6.20](#), [6.23](#), [6.24](#), [6.25](#), [6.26](#), [6.29](#), [6.30](#), [6.31](#), [6.32](#), [6.35](#), [6.36](#), [6.39](#), [6.40](#), [6.43](#), [6.44](#), [6.47](#), [6.48](#), [6.51](#), [6.52](#), [6.53](#), [6.54](#), [6.57](#), [6.58](#), [6.59](#), [6.60](#), [6.61](#), [6.62](#), [6.63](#), [6.66](#), [6.67](#), [6.68](#), [6.71](#), [6.72](#), [6.73](#), [6.74](#), [6.77](#), [6.78](#), [6.81](#), [6.82](#), [6.83](#), [6.84](#), [6.85](#), [6.86](#), [6.87](#), [6.88](#), [6.91](#), [6.92](#).

Search / Platform propositions/theorems: Propositions [6.8](#), [6.14](#), [6.21](#), [6.27](#), [6.33](#), [6.37](#), [6.41](#), [6.45](#), [6.49](#), [6.55](#), [6.64](#), [6.69](#), [6.75](#), [6.79](#), [6.89](#), [6.93](#), [6.94](#), Theorems [6.96](#), [6.98](#), [6.100](#).

Operational cautions: Remarks [6.1](#), [6.2](#), [6.3](#), [6.9](#), [6.15](#), [6.22](#), [6.28](#), [6.34](#), [6.38](#), [6.42](#), [6.46](#), [6.50](#), [6.56](#), [6.65](#), [6.70](#), [6.76](#), [6.80](#), [6.90](#), [6.95](#), [6.97](#), [6.99](#).

Explicit exclusions: Section [Z.21](#).

Part closure: Theorem [6.96](#).

Final non-confusion principle: Theorem [6.98](#).